

Cursor IDE ve GitHub Copilot for Business Kapsamlı Karşılaştırması

Giriş

Bir yazılım geliştirme ekibi olarak Visual Studio Code (VS Code) ortamında GitHub Copilot'u kullanmaya alıştık. Ancak son dönemde yapay zekâ destekli **Cursor IDE**'nin popülerliği arttı ve ekibimiz için uygun olup olmayacağını değerlendirmek istiyoruz. Bu raporda, **Cursor IDE (ekip lisansı)** ile **GitHub Copilot for Business** ürünlerini kapsamlı şekilde karşılaştırıyoruz. Karşılaştırma; fonksiyonellik, üretkenlik, güçlü/zayıf yönler, ekip odaklı kullanım, uzun vadeli getiriler, kurumsal özellikler ve lisanslama maliyeti gibi başlıklar altında yapılmıştır. Her iki ürün için detaylı teknik ve deneyimsel değerlendirmeler, doğrudan karşılaştırmalı tablolar, SWOT analizleri ve kurumsal karar vermeyi kolaylaştıracak özet bir öneri sunulmuştur.

Cursor IDE vs VS Code: Fonksiyonel Eşdeğerlik ve Geçiş Maliyeti

Cursor IDE, ek bir editör olarak değil, doğrudan VS Code'un yerine geçmeyi hedefleyen bir geliştirme ortamıdır. Nitekim Cursor, VS Code'un açık kaynak sürümünden çatallanan (fork) bir IDE'dir ve arayüzü ile temel özellikleri VS Code'a çok benzer ¹. Bu sayede VS Code'a alışkın geliştiriciler için arayüz ve editör deneyimi tanıdık olacaktır. Cursor, VS Code uzantılarını (extensions) destekler ve mevcut VS Code eklentilerinizi, temasınızı, kısayol tuşu ayarlarınızı Cursor'a içeri aktarabilirsiniz ¹. Örneğin, Cursor dokümantasyonu VS Code'daki profilin eklentiler, temalar, ayarlar ve kısayollarıyla beraber taşınabildiğini belirtmektedir. Bu sayede **fonksiyonel anlamda VS Code ile büyük ölçüde eşdeğer** bir IDE elde edilir ve mevcut iş akışlarınızı büyük ölçüde koruyabilirsiniz.

Buna karşın, Cursor'a geçiş **ciddi bir entegrasyon ve adaptasyon süreci** gerektirebilir. Kurulum ve geçiş adımları arasında Cursor istemcisinin tüm ekiplere dağıtılması, uzantı uyumluluğunun kontrolü ve geliştiricilerin yeni araçla çalışma biçimine alışması gibi maddeler bulunur ² ³. Enterprise kullanım senaryolarında yapılan bir analiz, **her bir geliştiricinin Cursor'a tam geçişi için 2-4 saatlik kurulum ve profil ayarlaması gerektiğini**, ayrıca **ekibin yeni ortamda verimliliğe ulaşması için ~1-2 haftalık** bir alışma dönemi olabileceğini ortaya koymuştur ⁴ ⁵. 40 kişilik bir ekip düşünüldüğünde, bu adaptasyon süreci ilk etapta **80-160 saatlik toplam üretkenlik kaybı** anlamına gelebilir ⁶. Diğer yandan, GitHub Copilot for Business kurulumu hali hazırda kullanılan VS Code'a bir eklenti yüklemek kadar kolaydır ve merkezi yönetim ile tüm ekibe dakikalar içinde açılabilir ⁷ ⁸. Copilot kullanımına başlamak genellikle kişi başı birkaç dakikada mümkün olurken, Cursor'da her geliştiriciye birkaç saat ayırmak gerekebilir. **Kurulum ve geçiş kolaylığı açısından Copilot'un bariz bir avantajı** vardır ⁸.

Geçiş maliyeti sadece zaman kaybıyla sınırlı değildir. Cursor şu an için VS Code'un tüm özelliklerini desteklese de (örneğin uzaktan geliştirme, hata ayıklama, kaynak kontrolü entegrasyonları VS Code tabanlı olduğu için mevcuttur), temel alınan VS Code sürümü zaman zaman geride kalabilir ve bazı yeni VS Code özelliklerine/eklenti güncellemelerine geç uyum sağlayabilir. Örneğin Cursor geliştirici topluluğunda, Cursor'ın VS Code çekirdeğinin güncellenmesinde gecikmeler olabildiği tartışılmıştır. Ayrıca ekipteki herkesin mevcut alışkanlıklarını (klavye kısayolları, eklenti ekosistemi kullanımı, vs.) yeni IDE'ye uyarlaması gerekir. Bu, kısa vadede üretkenliği etkileyebilir: Bir değerlendirmede, VS Code

kısayollarına alışkın deneyimli geliştiricilerin bile Cursor'da **ilk 1-2 hafta eski hızlarına ulaşmakta zorlandıkları** raporlanmıştır ⁵ .

Özetle, **Cursor IDE fonksiyonel olarak VS Code'un büyük kısmını karşılamakta ve AI destekli ek yetenekler sunmaktadır**. Ancak **bir geçiş planlandığında, hem eğitim/alışma sürecinin hem de geçiş süresince oluşacak üretkenlik kaybının maliyeti dikkate alınmalıdır**. Copilot for Business ise mevcut VS Code ortamına eklenti olduğu için **geçiş maliyeti neredeyse sıfırdır** – halihazırda VS Code kullanan bir ekibin araç setine Copilot'u eklemesi anlık bir işlemdir. Eğer ekibiniz Cursor'a özel bir avantajdan faydalanmayacaksa, yeni bir IDE'ye toplu geçişin yaratacağı kesintiyi ve maliyeti göz önünde bulundurmak önemlidir.

Üretkenlik: Copilot ve Cursor'un Entegre Yapay Zekâ Ajanlarının Karşılaştırması

Her iki ürün de geliştiricilerin daha hızlı kod yazmasını, rutin işleri otomatikleştirmesini ve karmaşık görevlerde yapay zekâ desteği almasını amaçlar. Ancak bunu gerçekleştirme yaklaşımları ve **üretkenliğe etkileri** farklılık gösterebilir.

GitHub Copilot, başlangıçta bir "tamamlayıcı" olarak tasarlandı – kod yazarken anlık olarak satır içi öneriler vererek hız kazandırmayı hedefliyor. Microsoft ve GitHub'ın paylaştığı verilere göre Copilot kullanımının görev tamamlama hızında %55'e varan iyileşme sağladığı ve önerilerin yaklaşık %30'unun geliştiriciler tarafından kabul edildiği görülmüş ⁹ . Ayrıca kullanıcı memnuniyeti de oldukça yüksek (%90'ların üzerinde) raporlanmıştır ¹⁰ . Bu hızlı kazanımlar, Copilot'un özellikle basit fonksiyonları hızlıca yazma, şablon kod üretme ve tekrarlayan kod parçalarını tamamlama konularında çok etkili olmasından kaynaklanır. Yani Copilot'u kullanmaya başlar başlamaz geliştiriciler, küçük ölçekli görevlerde hemen bir verim artışı hissedebilirler. Örneğin Copilot, bir fonksiyon bloğunun yalnızca imzasını yazdığınızda gerisini mantıklı bir şekilde tamamlayabilir veya bir sonraki satırı tahmin ederek kod akışını hızlandırır. Bu sayede, kısa vadede bireysel geliştirici üretkenliğinde gözle görülür artışlar yakalanabilir.

Cursor IDE ise "AI-first" (yapay zekâ öncelikli) bir çalışma düzeni benimsemeyi önerir ¹¹ . Cursor içinde birden fazla entegre **AI aracı (ajan)** bulunur: Kod tamamlama yanında, proje genelini anlayan sohbet tabanlı bir yardımcı, çoklu dosya düzenlemelerini yapabilen *Composer* özelliği, ve hata bulup çözebilen *Bugbot* gibi. Cursor, tek tek satır önerisinden ziyade, **doğal dil komutlarıyla birden fazla adımı gerçekleştiren otonom kod ajanı** fikrine odaklanmıştır ¹² ¹³ . Örneğin "şu projede X özelliğini ekle" dediğinizde, Cursor Agent (özellikle Claude modelini kullanarak) ilgili dosyaları bulup gerekli kodu birden fazla dosyada oluşturabilir, terminal komutlarını çalıştırabilir ve testleri geçene dek döngüsel olarak düzeltebilir ¹⁴ ¹⁵ . Bu yaklaşım, **daha büyük çaplı görevleri** (bir özelliğin baştan uca eklenmesi, kod tabanının yeniden düzenlenmesi gibi) otomatikleştirmede etkilidir. Cursor, bir bakıma geliştirme sürecini sohbet odaklı ve çok dosyalı bir akışa çevirmeyi hedefler: Geliştirici kod yerine isteğini yazar, AI arkaplanda kodu yazar ve gerekirse test edip düzeltir ¹⁶ ¹⁵ .

Bu farklı yaklaşımın üretkenliğe etkisi, **kısa vadede değil uzun vadede** ortaya çıkar. İlk defa Cursor kullanmaya başlayan bir ekip, Copilot'taki gibi anında %50 zaman kazanımı görmeyebilir. Hatta alışma sürecinde üretkenlik geçici olarak düşebilir (önceki bölümde bahsedilen adaptasyon süreci nedeniyle). Bir rapora göre, **ekipler Cursor'dan maksimum verimi haftalar içinde, AI ile çalışma pratiklerini benimsedikçe alıyor**; yani Cursor'ın değer önerisi birkaç günlük değil, birkaç haftalık bir öğrenme eğrisinin ardından tam olarak hissediliyor ¹⁷ . Bu nedenle, "*Copilot yazdığım kodu hemen hızlandırdı ama Cursor'ın katkısı biraz daha belirsiz başladı*" şeklinde deneyimler olabiliyor. Cursor'ın sohbet tabanlı ve proje genelini kapsayan özelliğine alışan ekipler ise daha karmaşık işler yapabildiklerini rapor ediyorlar. Örneğin Cursor ile **birden fazla dosyaya yayılmış büyük bir refaktör işlemi veya yeni bir modül**

ekleme çok hızlı yapılabilir, zira araç proje yapısını anlıyor ve tek komutla birçok dosyayı güncelleyebilir ¹⁸ ¹⁹ . GitHub Copilot da yakın zamanda “**Copilot Edits**” ve “**Copilot Agent Mode**” gibi özelliklerle benzer çoklu dosya düzenleme ve otonom görev tamamlama yetenekleri sunmaya başladı. VS Code içinde *Edits* özelliği, seçilen bir dosya grubunda istenen değişiklikleri tarayıp uygulayabilir; ancak yapılan testler Copilot Edits’in bazen yavaş kaldığını veya hatalı dosyaları değiştirebildiğini gösterdi ²⁰ ²¹ . Yine de, Copilot geliştikçe Cursor’ın önceleri benzersiz görülen ajan yeteneklerine yetişmeye başlamıştır. Nitekim 2025 yılı itibarıyla GitHub Copilot’un da VS Code içerisinde **çok adımlı “agent mode”** özelliğini ön izleme olarak sunmasıyla, **iki aracın fonksiyonel açıdan büyük ölçüde eşitlendiği** kullanıcılarca belirtilmiştir ²² .

Üretkenlik performansı açısından somut verilere bakarsak: Copilot’un öneri kabul oranları ve hız kazandırma istatistikleri iyi belgelenmiştir ⁹ , Cursor cephesinde ise şirket içi ölçümlerden bazı çarpıcı sonuçlar paylaşılmıştır. Örneğin Monday.com ekibinde Cursor kullanımıyla yeni ekip üyelerinin karmaşık depo içinde ramp-up (alışma) süresinin haftalardan günlere indiği belirtiliyor ²³ . Upwork’te Cursor kullanan bir ekibin, Pull Request (PR) hacminde %25 artış ve ortalama PR boyutunda %100 artış (yani daha fazla kod yazımı) görerek toplamda ~%50 daha fazla kod sevk ettikleri rapor edilmiş ²⁴ . Bu, Cursor ile mühendislerin daha yüksek hacimde iş çıkarabildiğine dair bir işaret olarak sunuluyor. Benzer şekilde Coinbase CEO’su, 2025 başlarında her mühendisin Cursor kullanmaya başlamasıyla **aylar süren proje işlerinin günlere indiğini** ifade ediyor ²⁵ . Bu tür geri bildirimler Cursor’ın potansiyel üretkenlik artışının altını çizse de, bunların genellikle *Cursor’ı iyice benimseyen* ekiplerden geldiğini unutmamak gerekir.

Kullanıcı geri bildirimleri iki ürünün pratikteki üretkenlik farkını anlamak için değerlidir. Bir Reddit kullanıcısı, **Cursor’daki Claude tabanlı modelin (ör. Claude “Sonnet” 3.5)** bazen çok akıllı, bazen de beklenmedik derecede yetersiz kalabildiğini; buna karşın Copilot’un (özellikle GPT-3.5 tabanlı modelinin) daha **tutarlı bir performans** sergilediğini belirtmiştir ²⁶ . Aynı kullanıcı, Cursor’da “hızlı istek” kotasının (500 adet/ay) yoğun kullanımda bir haftada dolduğunu, Copilot’ta ise benzer yoğunlukta kullanımda bile herhangi bir sınıra takılmadığını deneyimlemiştir ²⁶ . (Not: Copilot Business planının da her kullanıcı için aylık **300 adet “premium” istek** sınırı vardır ancak standart kod tamamlama için pratikte bu limite çoğu kullanıcı ulaşmamaktadır ²⁷ .) Ayrıca Copilot’un son model güncellemeleriyle **daha büyük bağlam penceresi** sunduğu, yani tek seferde daha fazla kod bağlamını işleyebildiği, Cursor’ın ise halihazırda daha sınırlı bir bağlam penceresi kullandığı ifade ediliyor ²⁸ . Bu durum özellikle çok büyük kod tabanlarında veya uzun dosyalarda çalışırken Copilot’a avantaj sağlayabilir.

Özetlemek gerekirse, kısa vadeli ve anlık üretkenlik artışında GitHub Copilot hızlı sonuç verip geliştiricilerin eli ayağı olurken, **Cursor daha stratejik ve derinlemesine bir üretkenlik artışı** vaat ediyor. Copilot ile ilk günden %10-20 hızlanmak mümkünken, Cursor ile belki ilk haftalarda benzer bir ivme yakalamak geliştirici alışkanlıklarına bağlı oluyor. Ancak uzun vadede Cursor doğru kullanıldığında daha büyük görevleri otomatikleştirerek bir geliştiricinin kapasitesini çarpanla artırabilir. Yine de güncel gelişmelerle birlikte Copilot, Chat ve Agent modülleriyle Cursor’ın birçok ayrıcalıklı özelliğini yakalamış durumdadır ²² . **Üretkenlik açısından “hangisi üstün” sorusunun net bir cevabı yoktur** – ekip kültürüne ve kullanım senaryosuna göre değişir. Copilot, hızlı **kazanımlar ve anlık verim artışı** sağlarken ⁹ , Cursor **alışma sonrası daha bütüncül ve büyük ölçekli işlerde** avantaj sağlayabilir ¹⁸ ¹² .

Her İki Ürünün Güçlü ve Zayıf Yönleri

Bu bölümde Cursor IDE ve GitHub Copilot’un öne çıkan güçlü yanları ile olası zayıflıklarını ayrı ayrı ele alıyoruz. Değerlendirmeler, ürün dokümantasyonlarına, bağımsız karşılaştırma yazılarına ve kullanıcı geri bildirimlerine dayanmaktadır.

Cursor IDE – Güçlü Yönler

- **Derin AI Entegrasyonu ve AI-Öncelikli Tasarım:** Cursor, en baştan yapay zekâyı merkeze koyarak tasarlanmıştır. Editör deneyimi içinde Chat tabanlı yardımcılar, çoklu dosya işlemleri, otomatik hata düzeltici (Bugbot) gibi araçlar gömülüdür. Bu sayede AI, Cursor'da ayrı bir araç değil, geliştirmenin doğal bir parçası haline gelir. Örneğin Cursor'da sohbet arayüzü kod bağlamını “birinci sınıf vatandaş” olarak görür ve klasörleri/dosyaları sürükleyerek sohbete eklemek bile mümkündür ¹⁹. Çok dosyalı değişiklik yapma, doğal dil ile yeni bileşenler oluşturma gibi gelişmiş üretkenlik özelliklerinde Cursor öndedir. Bu **“AI-first” çalışma biçimi**, özellikle karmaşık kod tabanlarında verimliliği artırır ²⁹.
- **Tüm Proje Kapsamında Bağlam ve Kod Anlayışı:** Cursor, projenin tamamını kavrayabilen bir tamamlayıcı sunar. Sadece açık dosyaları değil, proje yapısını, diğer dosyaları ve bağımlılıkları da göz önünde bulundurarak kod önerileri yapabilir ³⁰. *Composer* aracı, bir komutla tüm proje boyunca ilgili dosyalara yayılmış değişiklikler gerçekleştirebilir. Örneğin bir sınıfı birden çok dosyaya bölme veya bir özelliği eklemek için yeni modüler dosyalar oluşturma gibi işlemleri *otomatik* yapabilir ³¹. Bu proje-geneli farkındalık, Cursor'a büyük refaktörler ve kod tabanı evrimi konularında üstünlük sağlar. Geliştirici deneyimi açısından, Cursor kullanırken sık sık dosya arası geçiş yapmak veya tüm proje içinde arama yapıp manuel düzenleme yapmak yerine, AI'nın bu bütüncül görüşünden yararlanılabilir.
- **Model Esnekliği ve Güçlü AI Modellerine Erişim:** Cursor, arka planda birden fazla yapay zekâ modelini destekler ve kullanıcıların/organizasyonların tercihlerine göre seçim yapmasına olanak tanır. Örneğin OpenAI'nin GPT-4 (ve potansiyel olarak GPT-5) modelleri, Anthropic Claude 2/ Sonnet, Google Gemini gibi **“frontier” seviye modeller** Cursor içinde kullanılabilir durumdadır ³² ³³. Ekipler, ihtiyaçlarına göre daha hızlı bir model (ör. kendi “cursor-small” modeli veya GPT-3.5 türevi) ya da daha güçlü bir model (GPT-4, Claude 2 gibi) seçebilirler. Hatta Cursor'da organizasyonlar için *model erişim politikaları* tanımlanabilir: hangi durumda hangi modelin kullanılacağı, hangi modelin yasaklanacağı gibi merkezi kontroller uygulanabilir ³⁴. Bu esneklik, hem maliyet optimizasyonu (gereksiz yere pahalı modele çağrı yapılmaması) hem de performans optimizasyonu (ihtiyaç anında en iyi modelin kullanılması) sağlar. Copilot cephesinde de model seçenekleri artmış olsa da (Copilot da artık Claude 3.5 veya GPT-4 tabanlı farklı modlar sunuyor) ³⁵ ³⁶, Cursor bu konuda daha bağımsız ve özelleştirilebilir bir izlenim vermektedir.
- **Kurumsal Özelleştirme ve Kontrol:** Cursor, ekiplerin AI davranışını özelleştirmesine olanak tanıyan kurallar ve yapılandırmalar sunar. Örneğin projeye özel bir *.cursorrules* dosyası ile AI asistanının kod stiline dair uyması gereken kurallar veya yasaklı ifadeler belirtilebilir ³⁷. Organizasyon genelinde “Privacy Mode” adıyla bir ayar bulunur; bu aktifse Cursor, model isteklerinde hassas verileri göndermemek üzere filtre uygular ve kullanıcıların kazara şirket sırrı paylaşmasını engeller. Ayrıca **Role-Based Access Control (RBAC)** desteği ile kimlerin hangi AI özelliklerini kullanabileceği kademelendirilebilir ³⁸. Bu tür kontroller, büyük kurumlar için önemlidir. Copilot Business/Enterprise da benzer biçimde organizasyon geneli politikalar sunuyor olsa da (ör. “public code suggestions” engellenmesi, vs.) ³⁹, Cursor'ın odaklı bir şekilde bu özellikleri ürünün parçası yapması kurumsal müşterilerce takdir edilmektedir.
- **Veri Gizliliği ve Güvenlik Vaatleri:** Cursor, bulut tabanlı bir AI aracı olmasına rağmen kurumsal güvenlik ihtiyaçlarına güçlü biçimde vurgu yapar. **“Zero data retention” (sıfır veri tutulumu)** politikası sayesinde kullanıcı kodlarının Cursor veya arka plandaki LLM sağlayıcıları tarafından kalıcı olarak depolanmayacağını taahhüt eder ⁴⁰. Yani kodunuz, öneri üretildikten sonra ne Cursor ne de OpenAI/Anthropic tarafından saklanmaz ve AI modellerini eğitmek için kullanılmaz (Copilot da benzer şekilde, Business planlarında kullanıcı kodunu model eğitimi için

kullanmamaktadır). Cursor ayrıca SOC 2 Type II sertifikalıdır ve GDPR, CCPA gibi yasal düzenlemelere uyumluluğunu beyan etmektedir ⁴¹ ⁴² . Hassas sektörler için isteğe bağlı *on-premise/VPC* kurulum seçenekleri de Cursor tarafında sorgulanan bir özelliktir (mevcut olmasa da yol haritasında olabilir). Kısacası, Cursor kurumsal güvenlikle ilgili soru işaretlerini en aza indirmeye çalışır ve bu da büyük şirketlerin ürünü denemesini kolaylaştırır.

- **Kullanıcı Memnuniyeti ve Benimsenme:** Cursor henüz yeni sayılabilecek bir ürün olmasına karşın büyük teknoloji şirketleri arasında hızlı bir benimsenme yakaladığını iddia etmektedir. Resmi web sitesinde Samsung, OpenAI, Adobe, Stripe, Shopify, eBay, Nvidia, Coinbase gibi devlerin Cursor kullandığı belirtilmekte ve bazı yöneticilerin olumlu geri bildirimlerine yer verilmektedir ⁴³ ⁴⁴ . Örneğin Stripe CEO'su, binlerce mühendisin Cursor kullanmaya başlamasıyla verimliliklerinde çarpıcı artışlar gördüklerini ifade etmiştir ⁴⁴ . Yine bir iç değerlendirmede, mühendislerin %93'ünün "Cursor'ı tercih ettiği" rapor edilmiştir ⁴⁵ . Bu tür referanslar, Cursor'ın gerçek dünyada mühendislerce sevildiğini ve güçlü bir ürün deneyimi sunduğunu gösteriyor.

Cursor IDE – Zayıf Yönler

- **Yüksek Maliyet ve Kullanıma Göre Ücretlendirme:** Cursor, bireysel planlarda dahi GitHub Copilot'a kıyasla daha pahalı bir hizmettir. Ekip planı aylık kullanıcı başına **\$40** olarak fiyatlandırılmıştır (yıllık ödemede ~\$32) ⁴⁶ ⁴⁷ . Bu, Copilot Business'ın \$19/kışı/ay ücretinin iki katından fazladır. Dahası, Cursor'ın ücretlendirmesi "**kullanım tabanlı**"dır: Her kullanıcıya aylık belirli sayıda AI "istek" hakkı tanımlanmıştır (Teams planında muhtemelen Pro planıyla aynı, ~500 hızlı istek). Eğer geliştiriciler bu limitleri aşacak şekilde yoğun kullanırlarsa ek ücret yansıyabilir. Nitekim bazı kullanıcılar, Cursor Pro planında 500 isteği aşınca aylık faturanın \$44+ düzeyine çıktığını, Copilot'ta ise sınırsız kullanımda bile sabit \$10 (Pro için) ödendiğini belirtmiştir ⁴⁸ . Kurumsal planda Cursor "kullanım havuzlama" sunuyor ve esnek limitler tanıyabiliyor olsa da, **toplam sahip olma maliyeti (TCO)** açısından Cursor genelde daha maliyetli bir yatırım olacaktır (aşağıdaki lisanslama bölümünde detaylandırıldı). Özellikle halihazırda Copilot'a bütçe ayırmış ekipler için, Cursor'a geçiş bütçeyi önemli ölçüde artırabilir.
- **Geçiş ve Öğrenme Eğrisi:** Yukarıda değindiğimiz gibi, Cursor'a toplu geçiş ekibe bir öğrenme eğrisi ve adaptasyon maliyeti bindirir. VS Code'dan Cursor'a geçmek teknik olarak kolay olsa da, ekiplerin yeni AI-odaklı iş akışına alışması zaman alır. Bu süreçte geçici bir verim kaybı yaşanabilir. Ayrıca geliştiricilerin alıştığı bazı VS Code eklentilerinin Cursor ortamında %100 aynı çalışmaması veya küçük uyumsuzluklar göstermesi ihtimali vardır. Örneğin Cursor, Microsoft'un kapalı kaynak VS Code Market'ı yerine OpenVSX kullanıyor olabilir; bu da bazı eklentilerin bulunabilirliğini etkileyebilir. **Kısacası, Cursor'ın en güçlü yönü olan "AI-first" yaklaşım, aynı zamanda ekip için bir değişim yönetimi zorluğu** yaratır. Bu zayıflık, değişime dirençli veya yoğun teslimat takvimine sahip ekipler için önemli olabilir.
- **Tutarsız Model Davranışı ve Bağımlılık:** Cursor'ın performansı büyük ölçüde entegre ettiği üçüncü parti AI modellerine bağlıdır (OpenAI, Anthropic vb.). Özellikle bugünkü haliyle Cursor'ın gelişmiş Agent modu sadece Anthropic Claude modeliyle çalışmaktadır ¹² ¹³ . Kullanıcılar, Claude tabanlı önerilerin kalitesinin zaman zaman değişebildiğini, bazı günler "çok akıllı" davranan modelin bazen de basit hatalar yapabildiğini raporlamıştır ²⁶ . Bunun sebepleri arasında modelin kendisindeki tutarsızlıklar veya API kısıtlamaları sayılabilir. Örneğin bir kullanıcı, Cursor'ın kullandığı Anthropic API'nin yoğun kullanım saatlerinde kısıntıya gidip cevapları geciktirdiğini veya basitleştirdiğini gözlemlemiştir ⁴⁹ . Copilot ise Microsoft/GitHub altyapısında tutarlılık ve servis düzeyi konusunda daha istikrarlı görünüyor (GitHub, Copilot için Amazon Bedrock altyapısını ve kendi optimizasyonlarını kullanıyor olabilir ve hizmet kalitesini garanti

ediyor) ⁴⁹. Sonuç olarak Cursor, dış modellerin performansına bağımlıdır ve bazen **öngörülemez performans dalgalanmaları** yaşanabilir. Bu da bir zayıflık olarak not edilmelidir.

- **Yeni/Genç Ürün ve Olgunluk:** Cursor, 2023 itibarıyla piyasaya çıkmış, hızla gelişen bir üründür. Henüz olgunlaşma sürecinin erken evrelerindedir. Bu durum, sık sık yeni özelliklerin eklenmesi (ve potansiyel bug'lar), hızlı sürüm değişiklikleri ve kararlılık konularında soru işaretleri anlamına gelebilir. Copilot ise 2021'den beri geliştirilen, milyonlarca kullanıcıdan geri bildirim almış nispeten daha olgun bir hizmettir. Cursor'da karşılaşılabilecek beklenmedik hatalar, performans sorunları veya eksik dokümantasyon durumları olasılık dahilindedir (nitekim bir kaynak, Cursor'ın kurumsal politika yönetimi API'leri konusunda belgelendirme eksikliği olduğunu vurgulamıştır ² ⁵⁰). Ayrıca Cursor arkasındaki Anysphere firmasının uzun vadeli istikrarı da çok büyük şirketlere kıyasla daha belirsiz olabilir. Büyük bir şirket için ürünün sürdürülebilirliği ve sürekli destek alabilmek kritik olduğundan, yeni bir startup ürününe kurumsal bağlamda güvenmek bir miktar risk içerir.

- **Kaynak Kullanımı ve Performans İhtiyaçları:** Cursor IDE, entegre AI özellikleri nedeniyle VS Code'a kıyasla daha yüksek sistem kaynağı tüketebilir. Örneğin Cursor, tam verimli çalışması için 8 GB RAM öneriyor ki bu VS Code'un temel gereksinimlerinin üzerindedir ⁵¹. Özellikle aynı anda büyük modellerle etkileşim, arka planda kod taraması gibi işlemler yaparken donanım üzerindeki yük artabilir. Geliştiricilerin makinalarının bu ek yükü kaldırabildiğinden emin olmak gerekir. Copilot ise mevcut editör üzerinde küçük bir eklenti olduğu için ekstra bir yük getirmez; bulut tarafında çalışır ve geliştiricinin bilgisayarında fark edilir bir performans maliyeti yaratmaz.

GitHub Copilot for Business – Güçlü Yönler

- **Kolay Entegrasyon ve Mevcut Altyapıyla Uyum:** Copilot, halihazırda kullandığımız VS Code'a dakikalar içinde entegre edilebilir bir hizmettir ⁵² ⁶. Herhangi bir yeni IDE öğrenmeye gerek kalmaz; geliştiriciler mevcut araçlarında (VS Code, Visual Studio, JetBrains IDE'leri, Neovim vb.) Copilot'u kullanabilirler ⁵³ ⁵⁴. Özellikle ekibimiz zaten VS Code üzerinde olduğundan, Copilot *"altyapıyı olduğu gibi kullanma"* avantajına sahiptir. Bir yönetici açısından bakıldığında, Copilot'u ekibe yaymak *"tek tıkla"* halledilebilecek bir iştir: GitHub organizasyon ayarlarından lisanslar atanır ve geliştiriciler VS Code Marketplace üzerinden eklentiye yükler, hepsi bu ⁷. Bu kullanım kolaylığı, ekiplerin hemen fayda görmeye başlamasını sağlar ve geçiş bariyerlerini ortadan kaldırır. Sonuç olarak **Copilot, mevcut iş akışını bozmadan AI gücünü ekleyebilmesiyle** çok güçlüdür.

- **Kanıtlanmış Üretkenlik ve Yüksek Kullanıcı Memnuniyeti:** Copilot, milyonlarca yazılımcı tarafından denenmiş ve geniş çapta benimsenmiştir. 2023 Stack Overflow geliştirici anketinde, katılımcıların %44'ünün halihazırda bir AI kod yardımcısı kullandığı veya kullanmayı planladığı belirtilmişti ve bunların büyük bölümünde Copilot öne çıkıyordu ⁵⁵. Microsoft'un yaptığı kontrollü deneylerde Copilot kullanan geliştiricilerin belirli görevleri **%55 daha hızlı tamamladığı** raporlanmıştır ⁵⁶. Gerçek dünya kullanımında da geliştiriciler Copilot'un özellikle rutin işleri hızlandırdığı, *"boilerplate"* kod yazmayı devraldığı, hata riskini azalttığı ve genel olarak **daha üretken hissettirdiğini** dile getirmektedir. %90'lara varan memnuniyet oranları ⁹ Copilot'un büyük oranda geliştiricilerin beğenisini kazandığını gösteriyor. Bu düzeyde yaygın kullanım ve pozitif geri bildirim, Copilot'un güçlü bir ürün olduğunu ispatlar niteliktedir.

- **Geniş Platform ve Dil Desteği:** Copilot yalnızca VS Code ile sınırlı olmayıp, JetBrains IntelliJ serisi, Neovim, Visual Studio, Azure Data Studio, hatta çeşitli IDE'lere ve terminallere entegre edilebilir ⁵⁴ ⁵⁷. Bu, ekibinizde farklı geliştirme ortamları kullanan kişiler olsa bile hepsinin Copilot'tan faydalanabileceği anlamına gelir. Örneğin arka uç ekibi VS Code'da, mobil ekip Android Studio

(IntelliJ tabanlı) kullanıyorsa bile Copilot ortak bir AI asistanı çözümü sağlar. Dil desteği açısından da Copilot, OpenAI'nin evrensel modellerini kullandığı için neredeyse **her popüler programlama dilinde** fayda sağlayabilir (Python, JavaScript/TypeScript, Java, C#, Go, PHP, Ruby, vs. dahil). JetBrains tarafından yapılan bir araştırma, Copilot'un poliglot bir araç olarak **çeşitli dil ve çerçevelere hızla adapte olabildiğini** ortaya koymuştur ⁵⁸. Cursor ise VS Code ekosistemini devraldığı için dil desteği geniş olsa da, Copilot kadar farklı IDE ortamlarına nüfuz etmiş durumda değildir.

- **GitHub Ekosistemi ile Sıkı Entegrasyon:** Copilot, GitHub platformunun doğal bir uzantısı gibidir. Kurumsal sürümde **Pull Request asistanı**, otomatik açıklama oluşturma, test senaryoları önerme gibi GitHub üzerinde üretkenlik artırıcı özellikler devreye girmektedir. Örneğin bir PR açıldığında Copilot, değişiklikleri analiz ederek özet açıklama ve içerik önerebilir. Yine VS Code içerisinde *"Copilot Code Review"* özelliği ön izleme olarak sunulmuş olup, geliştiriciler kodlarını yerel olarak gözden geçirirken Copilot'un olası sorunları satır içi belirtmesini sağlayabilir ⁵⁹. Bu özellik, **code review** aşamasında ikinci bir çift göz gibi çalışarak ekip verimini artırır. Cursor cephesinde de benzer bir *Bugbot* olmasına rağmen, Copilot'un GitHub ile doğal bağlantısı (örneğin commit mesajlarını otomatik önermesi, GitHub Issues ve Discussions ile etkileşim gibi planlanan özellikler) bir avantajdır. Ayrıca Copilot'un CLI aracı, komut satırında bile AI yardımını (bash komutları oluşturma vb.) mümkün kılarak geliştiricinin tüm iş akışına nüfuz edebilir. Kısacası Copilot, GitHub ve çevresindeki araç zincirine entegre olarak **uçtan uca yazılım geliştirme döngüsünde** yardımcı olur.

- **Olgun Kurumsal Güvenlik ve Uyum (Compliance) Çerçevesi:** GitHub Copilot for Business, Microsoft ve GitHub'ın kurumsal güvenlik standartlarına uygun olarak geliştirilmiştir. 2024'te Copilot hizmeti için SOC 2 Type 1 raporu yayınlanmış, ISO 27001:2013 sertifikasyonu tamamlanmıştır ⁶⁰. Avrupa kullanıcıları için verilerinin ABD dışına çıkmaması adına **AB veri yerielliği** desteği getirilmiştir (GitHub Enterprise Cloud Dublin sunucuları aracılığıyla) ⁶¹. Kurumlar, Copilot'un nasıl kod önerdiğine dair ince ayar yapabilir: Örneğin şirket politikası olarak, Copilot önerilerinin açık kaynak kodlarla birebir eşleşme taşıması engellenebilir, böylece lisans riski minimize edilir ³⁹. Yine hassas içerikleri (belirli dosyalar, klasörler) Copilot'ın görmezden gelmesini sağlamak (Content Exclusion) Enterprise planında mümkündür ⁶². Copilot ayrıca **kapsamlı bir denetim (audit) ve günlükleme** sunar: Kurum yöneticileri hangi kullanıcının ne kadar öneri aldığını, kaç satırını kullandığını veya hangi dosyalarda AI yardımı aldığını (belirli bir seviyede) raporlayabilir; tüm aktiviteler audit log'lara işlenebilir ⁶³. Bu şeffaflık, regüle sektörlerde zorunlu olabilecek denetim süreçleri için kritiktir. Özetle **Copilot, kurumsal güvence anlamında daha "rahat uyku uyutan" bir seçenek** olarak görülebilir, zira büyük şirketlerin risk ve uyum beklentilerine dair kapsamlı bir çerçeve sunar ⁶⁴.

- **Maliyet Avantajı ve Tahmin Edilebilirlik:** Copilot, bireysel ve kurumsal kullanımda fiyat/performans dengesi yüksek bir üründür. Ekipler için Copilot Business ücreti kullanıcı başına aylık \$19'dır ⁶⁵, yıllık ödemede indirimli olarak kullanıcı başı \$16 civarına gelebilmektedir. Bu fiyat, Cursor Teams planının yaklaşık yarısıdır. Üstelik Copilot'ta kullanım kotası kavramı Cursor kadar ön planda değildir – "adil kullanım" politikası olsa da çoğu ekip üyesi için **sınırsız yakın bir deneyim** sunar. 2025 itibarıyla Copilot Business planı her kullanıcıya aylık **300 adet premium (örneğin GPT-4 gibi) istek** hakkı tanıyor olsa da, normal kod tamamlama sınırsızdır ve bu limit tipik bir yazılımcının ihtiyaç duyacağının oldukça üzerindedir ⁶⁶. Cursor'da ise 500 istek limiti daha hızlı dolabilir ve aşıldığında ek ücret gerektirebilir ⁶⁷. Bu bakımdan Copilot'un maliyeti **daha sabit ve öngörülebilir**. Bütçe planlaması yaparken Copilot için koltuk sayısı * birim fiyat şeklinde hesap yapmak kolayken, Cursor için olası ek kullanım maliyetlerini hesaba katmak gerekir. Ayrıca Copilot, öğrenciler, açık kaynak katkıları için ücretsiz veya indirimli planlar

sunarak topluluk içinde yaygınlaşmıştır. Sonuç olarak maliyet dezavantajı yaşamadan ekibin her üyesine AI yardımcısı vermek isteyen şirketler için Copilot güçlü bir seçimidir.

GitHub Copilot – Zayıf Yönler

- **Sınırlı Otonomluk (Gelişmekte Olan Ajan Özellikleri):** Copilot'un çekirdek ürünü, satır içi kod tamamlama ve basit chat önerileri üzerine kuruludur. Cursor'ın aksine, Copilot uzun süre tam otonom çok adımlı görev yürütme yeteneği sunmamıştı. Her ne kadar 2025'te **Copilot Agent Mode** duyurulmuş olsa da ²², bu özellik henüz ön izleme aşamasındadır ve herkes tarafından aktif şekilde kullanılmıyor olabilir. Yani *şu an itibarıyla*, Cursor'daki kadar "iş teslim edebileceğiniz" bir AI ajan deneyimi Copilot'ta tam oturmamış olabilir. Örneğin Cursor Agent komutu ile bir projeyi derleyip test edip hataları düzeltme akışı otomatize şekilde yapılabilirken ¹⁶ ¹⁵, Copilot'ta bu deneyim henüz yenidir ve kullanıcı müdahalesine daha çok ihtiyaç duyabilir. Ancak bu fark hızla kapanmaktadır; dolayısıyla bu zayıflık kalıcı olmayabilir.
- **Modeller ve Özelleştirme Üzerinde Kısıtlı Kontrol:** Copilot, GitHub'ın sunduğu bulut hizmeti olarak kutudan çıktığı şekliyle kullanılır. Sunulan AI modelleri ve parametreleri GitHub tarafından seçilir (OpenAI'nin belirli sürümleri, veya GitHub'ın kendi optimize modelleri gibi). Yakın zamanda model seçenekleri (GPT-4 tabanlı vs daha hızlı GPT-3.5 tabanlı vs Claude tabanlı) kullanıcı arayüzüne gelse de ³⁵, **kurumların kendi API anahtarlarını kullanması veya kendi özel modelini entegre etmesi Copilot ile mümkün değildir.** Cursor ise örneğin kendi OpenAI API anahtarını kullanıp model çağırmanıza izin verebiliyor (özellikle self-hosted planlarda) veya en azından model seçimi esnekliği sağlıyor. Copilot'ta *“.github/copilot-instructions.md”* dosyası ile kodlama stiline dair bazı yönlendirmeler yapabilseniz de ⁶⁸, genel olarak Cursor'ın *“.cursorrules”* kadar kapsamlı özelleştirme imkanı yoktur. Yani Copilot, **"kutuyu aç ve kullan"** yaklaşımı ile pratik olsa da, ileri düzey kullanıcılar için bir miktar kapalı kutu sayılır; AI'nin kararlarını ince ayar yapma imkanı kısıtlıdır.
- **Veri Gizliliği Algısı:** Copilot'un ilk çıktığı dönemde en çok tartışılan konulardan biri de eğitim aldığı veri setiydi. GitHub üzerindeki herkese açık depo kodlarını tarayarak model eğitildiği ve önerilerin bazen bu lisanslı kodlardan alıntılar içerdiği eleştirildi. Copilot for Business ile GitHub bu konularda iyileştirmeler yaptı – örneğin **"public code blocking"** özelliği sayesinde önerilen kod parçası internette bulunan bir kodla birebir eşleşiyorsa bunu tespit edip kullanıcıya bildirme veya engelleme seçeneği getirildi ³⁹. Ayrıca Business/Enterprise müşterilerinin kodlarının, modelin gelecekteki eğitim setlerine katılmayacağı taahhüt edildi. Yine de bazı kurumlardaki hukuk ve güvenlik birimleri, Copilot'un kapalı bir servis olmasından dolayı *"veri kontrolü tamamen bizde değil"* endişesini taşıyabilir. Örneğin **iç ağda, internet bağlantısı olmadan** kullanmak isteyen çok mahremiyet odaklı bir şirket için Copilot şu an çözüm sunmamaktadır (GitHub Enterprise Server sürümünde Copilot desteği yoktur ⁶⁹). Cursor da bulut temelli olsa da, en azından organizasyon içerisinde Privacy Mode, kendi API anahtarlarını kullanma gibi seçenekleriyle bu algıyı bir nebze daha iyi yönetebilir. Kısacası Copilot, **çok yüksek güvenlik gerektiren, kapalı devre ortamlarda** dezavantajlı konumdadır.
- **Ekip Kullanım Analitiklerinde Sınırlılık:** Copilot Enterprise, yöneticilere kullanım metrikleri ve audit log sağlasa da, bunlar halihazırda sınırlı bilgiler sunmaktadır. Örneğin bir yöneticinin Copilot kullanımından elde ettiği metrikler arasında kaç kullanıcı aktif, kaç öneri alınmış gibi temel ölçümler vardır; ancak Cursor'ın admin panelinde görülebilen detaylı kullanım istatistikleri (ekip bazında toplam token kullanımı, en çok AI kullanan geliştirici, vs.) gibisi henüz Copilot tarafında public arayüzde yoktur. Cursor Teams planında *"usage analytics dashboard"* ile AI kullanımının ekip içi dağılımı izlenebilmektedir ⁷⁰. GitHub Copilot ise haliyle GitHub platformuna entegre olduğundan, yöneticiler GitHub organizasyon ayarlarından basit etkinlik

raporlarına erişirler. Bu fark, her ne kadar belirleyici büyük bir etken değilse de, veri odaklı yönetim isteyen ekipler için dikkate alınabilir.

- **Lisanslama Ekosistemine Bağımlılık:** Copilot kullanmak isteyen bir şirket, GitHub platformunun ekosistemine de dahil olmak durumundadır. Copilot lisansları GitHub üzerinden yönetilir, kullanıcıların GitHub hesabı olması gerekir, vs. Bazı şirketler kodlarını GitHub yerine GitLab/Bitbucket gibi sistemlerde barındırıyor olabilir; bu durumda sadece Copilot kullanmak için GitHub ile entegrasyon kurmaları gerekecektir. Yine Microsoft hesabı/ürünleriyle bağ (Azure Active Directory entegrasyonu gibi) gündeme gelebilir. Bu durum çoğu için sorun olmasa da, **bazı şirketler tedarikçi kilitlenmesi (vendor lock-in)** konusunda hassastır. Cursor, bağımsız bir şirket olarak burada alternatif olmayı deniyor – örneğin kodlarınızı GitHub dışında saklıyor olsanız bile Cursor'ı IDE olarak kullanabilirsiniz. Kısaca Copilot, **GitHub ve Microsoft ekosistemine sıkı bir şekilde bağlıdır**, bu da esneklik açısından bir dezavantaj olarak görülebilir.

Not: Yukarıdaki güçlü/zayıf yönler genel durumu özetlemektedir. Aşağıdaki SWOT analizinde bu noktalar maddeler halinde tekrar özetlenecektir.

Ekip Odaklı Kullanım Özellikleri ve Entegrasyon Yetenekleri

Her iki ürün de bireysel geliştirici üretkenliğini artırmanın yanı sıra **ekip çapında verim ve işbirliği** hedefleyen özellikler içerir. Bu bölümde Copilot ve Cursor'ın ekip odaklı kullanım senaryolarındaki davranışlarını inceliyoruz.

Entegrasyon ve İş Akışı: GitHub Copilot, adından da anlaşılacağı üzere GitHub ile doğal bir entegrasyona sahiptir. Şirketiniz GitHub üzerinde iş akışları yürütüyorsa, Copilot bu süreçte sorunsuz eklenebilir. Örneğin GitHub üzerinde **Pull Request asistanı** sayesinde, bir PR açıldığında Copilot PR'ın içeriğini özetleyip açıklama taslağı verebilir ya da PR içindeki kod değişikliklerini analiz edip olası iyileştirme önerileri sunabilir. VS Code eklentisi olarak Copilot Chat, doğrudan kaynak kontrol panelinde PR'daki değişiklikleri gözden geçirme yeteneğine kavuşmuştur ⁵⁹. Geliştirici, değiştirdiği satırların yanına Copilot'un ürettiği yorumları tek tıkla ekleyebilir veya bunları düzeltebilir. Bu özellik henüz yeni olsa da, **kod inceleme süreçlerinde AI desteği** sağlayarak ekiplerin code review yükünü hafifletmeyi amaçlamaktadır. Cursor tarafında benzer bir işlevi **Cursor Bugbot** üstleniyor. Cursor; GitHub ile entegre bir bot olarak PR'larınızı tarayıp olası hataları listeler ve bunlara bir önem puanı atar. Ardından sizden onay alarak hataları otomatik düzeltebilir ⁷¹ ⁷². **Bugbot'un ekiplere faydası**, hem *knowledge sharing* (AI, herkese açık bir yazılımdaki bilinen hataya karşı uyarı verebilir, böylece tecrübeli bir kod gözünün yakalayacağı şeyi AI yakalar) sağlar, hem de inceleme süresini kısaltır. Ancak Bugbot'un **kullanımı isteğe bağlı bir eklenti** şeklindedir ve Cursor'ın ana paketine ek olarak ücretlendirilir (takım planı kullanıcı başı \$40/ay) ⁷³. Copilot'un PR ve code review asistanları ise ürünün parçası olarak sunulmaktadır.

Takım İçi Paylaşım ve İşbirliği: Cursor, bir IDE olduğu için takım odaklı bazı ilginç özellikler sunabilir. Örneğin *Cursor Web Agents* özelliği ile tarayıcı üzerinden de Cursor'un AI ajanına erişim sağlanabilir ⁷⁴. Bu, geliştirici olmayan ekip üyelerinin bile (örneğin QA ekibi veya proje yöneticileri) kod tabanı ile ilgili sorular sormak veya fonksiyonel bir sorgu yapmak için Cursor'ın AI'ını kullanabilmesi anlamına gelebilir – tabii ki bu oldukça yeni bir yaklaşım, ancak mümkün olduğu belirtiliyor. Ayrıca Cursor, yapay zekâ tarafından üretilen kod değişikliklerini *branch*'ler halinde yönetmeyi teşvik ediyor; bir örnekte, mühendislerin her gün bir düzine "agent branch" oluşturup bunları birleştirmesinin normalleştiği ve böylece paralelde birden fazla AI yardımıyla faydalandığı anlatılmış ⁷⁵. Bu aslında takım verimliliğine etki eden bir nokta: AI ajanları arka planda çeşitli işleri hallederken, geliştiriciler ana akışlarına devam edebiliyor, sonra ajanların ürettiği değişiklikleri code review yapıp birleştirebiliyorlar.

Sentry'den bir yöneticinin ifadesiyle, bu durum sürüm sıklığından bug çözme hızına kadar her şeye olumlu yansımış ⁷⁵ . Yani Cursor, ekibin her üyesinin yanında birer “çalışkan yardımcı asistan” varmışçasına çalışmasına ortam sağlıyor.

Öte yandan GitHub Copilot da **takım genelinde benimsenmeye** odaklı rehberlik sunuyor. GitHub, Copilot'un ekip içinde yaygınlaştırılması için yöneticilere özel “**usage metrics**” ve “**adoption trends**” raporları sağlar. Örneğin yöneticiler, şirket içinde kaç kişinin Copilot lisansını aktif kullandığını, kaç kişinin henüz kullanmaya başlamadığını görebilir; hatta “**inactive user reminder**” özelliğiyle Copilot kullanmayan geliştiricilere motive edici hatırlatma mesajları gönderebilirler ⁷⁶ . Bu tür özellikler, satın alınan lisansların atıl kalmaması ve ekip genelinde AI yardımı kullanımının teşvik edilmesi açısından değerlidir. Cursor'da da yönetici panelinden ekip üyelerinin kullanımı izlenebilir (kim ne kadar istek yapmış, vs.) ve bu veriler ışığında ek eğitim veya yönlendirme planlanabilir.

Eğitim ve Onboarding: Ekip odaklı bir diğer boyut, **yeni ekip üyelerinin eğitimi ve mevcut üyelerin gelişimidir**. AI araçları bu alanda da kullanılabilir. Cursor'ın “**Your engineering knowledge base**” sloganıyla vurguladığı şey, AI'nın kod tabanınızı ve dokümantasyonunuzu adeta bir bilgi üssü gibi kullanabilmesidir ⁷⁷ . Yeni gelen bir geliştirici, Cursor chat'e projenin belli bir modülünün ne işe yaradığını veya belirli bir fonksiyonun nerede kullanıldığını sorabilir; Cursor tüm kod tabanını tarayıp yanıt verebilir. Bu, mentörlük ihtiyacını bir miktar azaltır ve **onboarding süresini kısaltabilir**. Nitekim Monday.com'daki ekip lideri, önceden haftalar alan kod tabanını öğrenme sürecinin Cursor ile “daha yapılandırılmış bir sürece” dönüştüğünü belirtmiştir ²³ . Copilot'ta benzer bir yaklaşım “**GitHub Copilot İlişkili Bilgi Tabanları**” (Copilot Knowledge Bases) özelliğiyle gelmektedir ⁶³ . Enterprise planında, şirket dokümanları veya rehberleri Copilot Chat ile entegre edilebilir, böylece Copilot sorulan sorulara şirketin kendi dokümanlarına dayanarak yanıt verebilir. Bu özellik henüz yeni (2025 itibarıyla) ve tam olgunlaşmamış olabilir; ancak vizyon olarak Copilot da ekibin kurumsal hafızasını AI ile birleştirmeyi hedefliyor. Sonuç olarak her iki ürün de, yeni başlayanların öğrenme eğrisini azaltmaya ve tüm ekibin bilgi paylaşımını hızlandırmaya yardımcı araçlar barındırıyor.

Entegrasyon Yetenekleri: Cursor ve Copilot'un diğer araçlarla entegrasyonuna gelirse, Copilot burada biraz daha avantajlı çünkü Microsoft/GitHub ekosisteminin parçası. Örneğin **Azure DevOps** kullanan bir şirket, henüz Copilot desteği olmasa bile yakında entegre edebilir. Slack veya Microsoft Teams üzerinden Copilot ajanına soru sormak gibi özellikler GitHub'ın yol haritasında bulunmaktadır (dokümantasyonda Slack/Teams ile Copilot agent entegrasyonu için API örnekleri yer alıyor ⁷⁸). Cursor tarafında da forumlarda Slack entegrasyonu soruluyor; ancak resmi bir özellik var mı belirsiz.

Sonuç: Ekip perspektifinden bakıldığında, **her iki araç da takım verimliliğini artırma ve AI'ı bir ortak “takım üyesi” haline getirme yönünde ilerliyor**. Copilot, zaten var olan geliştirme platformlarınıza akıllıca nüfuz ederek işbirliğini geliştiriyor; Cursor ise yeni bir platform sunarak ekibin alışkanlıklarını değiştirmek pahasına daha derin bir AI entegrasyonu vadediyor. Takımınız yoğun şekilde GitHub üzerinde çalışıyor ve halihazırda entegre bir araç zinciriniz varsa, Copilot bu zincire kusursuz eklemlenir. Eğer takımınız AI'ı bütün süreçlerine yaymaya gönüllüyse ve kod yazımından test etmeye, dokümantasyondan refaktöre her adımda bir AI partneri istemekteyse, Cursor bu vizyona uygun araçlarla dolu bir dünya sunar.

Uzun Vadeli Getiriler: Ürün Geliştirme Süresi, Ekip Verimliliği ve Onboarding Etkileri

Yapay zekâ destekli bu araçların kısa vadeli kazanımları kadar, **uzun vadede sağlayabilecekleri stratejik getiriler** de değerlendirilmelidir. Bir teknoloji yatırımının geri dönüşü, sadece ilk birkaç

haftada ölçülen hızlanma değil, **aylar ve yıllar içinde ekip dinamizmine ve ürün kalitesine etkisi** ile anlaşılır.

Ürün Geliştirme Süresi ve Sürüm Döngüsü: Hem Cursor hem de Copilot, yazılım geliştirme döngüsünü hızlandırarak **pazara ürün sunma süresini (time-to-market)** kısaltabilir. Copilot'un getirdiği ivme daha mikro düzeydedir – her bir geliştirici günlük işlerinde zaman kazanır, bu da sprint sonunda biraz daha fazla iş bitmiş olmasıyla sonuçlanır. Microsoft'un bir çalışmasında, Copilot kullanan ekiplerin planlanan görevleri tamamlama hızının belirgin ölçüde arttığı gösterilmiştir ⁵⁶. Cursor ise yazılım geliştirmeye daha farklı bir yaklaşım getirdiğinden, uzun vadede daha radikal kazanımlar mümkün olabilir. Örneğin Cursor kullanan bir ekip, aynı anda birden çok özelliği paralel geliştirmeyi deneyebilir: İnsan geliştiriciler ana önemli işleri yaparken, Cursor Agent ikincil işleri halledebilir. Bir CTO'nun ifadesiyle Cursor, “ölçek çarpanı (scale-multiplier)” etkisi yaparak organizasyonun genel hızını artırmıştır ⁷⁹. Sentry'de yöneticilik yapan Cody De Arkland, Cursor sayesinde günde düzinelerce AI kaynaklı yan dal (branch) birleştirmenin normal hale geldiğini, bunun da sürüm sıklığını ve bug kapatma hızını ciddi oranda yükselttiğini paylaşıyor ⁷⁵. Bu anekdotlar, uzun vadede Cursor'ın doğru kullanıldığında şirketin inovasyon hızını katlayabileceğine işaret ediyor.

Ekip Verimliliği ve Mühendis Başına Üretim: Uzun vadede bir diğer önemli kriter, ekibin toplam çıktı miktarı ve her bir mühendisin “**verim katsayısı**”dır. AI araçlarının hedefi, geliştiricileri sıradan ve zaman alan işlerden kurtarıp daha yaratıcı veya kritik işlere odaklamaktır. Bu hem çıktı kalitesini yükseltir, hem de aynı sayıda mühendisle daha fazla iş yapılabilmesini sağlar. Coinbase örneğinde, CEO Brian Armstrong 2025'te tüm mühendislerin Cursor kullanmaya başlamasıyla “**tek bir mühendisin artık eskisinden çok daha büyük işleri tek başına halledebildiğini, aylar sürecek işleri günlere indirebildiğini**” ifade ediyor ²⁵. Bu oldukça çarpıcı bir verimlilik artışı anlatımıdır. Benzer şekilde Upwork'te bir ekip, Cursor sonrasında geliştirici başına düşen kod miktarının ve PR sayısının anlamlı şekilde arttığını paylaşmıştır ²⁴. Copilot tarafında ise bir GitHub blog yazısında, geliştiricilerin *haftalık ~4-8 saat* kazanabildiği, yani haftada bir tam günlerini Copilot sayesinde daha verimli geçirdikleri belirtilmişti ⁵⁶ ⁵⁸. JetBrains de kendi AI asistanları için **haftada 8 saat kazanç** (yani mühendis başına verim %20 artışı) gibi rakamlar telaffuz ediyor ⁵⁸. Bu rakamlar, Copilot ve benzeri araçların uzun vadede her bir çalışanın üretim kapasitesini artırdığını gösteriyor. Eşit sayıda mühendisle daha fazla özelliği tamamlayabilir, daha sık sürüm çıkarabilir veya aynı işi daha az insanla yaparak kaynak optimizasyonu sağlayabilirsiniz.

Onboarding (Yeni Geliştiricilerin İşe Alıştırılması): Yeni ekip üyelerini verimli kılmak, özellikle büyük kod tabanlarında zorlu bir süreçtir. Burada AI asistanları adeta bir *can simidi* olabilir. Cursor'ın geniş kod bağlamını anlayıp açıklayabilme özelliği, yeni gelenlerin “Bu kütüphane ne işe yarıyor?”, “Falanca modül nasıl çalışıyor?” gibi sorularına anında yanıt alabilmesini sağlar. Bu, daha az tecrübeli geliştiricilerin dahi karmaşık kodlarda özgüven kazanarak değişiklik yapabilmesi anlamına gelir. Monday.com'daki deneyimde, Cursor'ın devreye girmesiyle **haftalar süren ramp-up sürelerinin çok kısaldığı** vurgulanmıştır ²³. Ayrıca AI, 7/24 hazır bir eğitmen gibi kullanılabilir – yeni gelen biri geç saatte takıldığı bir sorunu Cursor Chat'e sorup yol alabilir, bu da kıdemli ekip üyelerine daha az yük bindirir. Copilot Chat de benzer şekilde bir kodu açıklama, ne yaptığını anlatma gibi konularda faydalı olabilir; ancak Cursor kadar projenin tamamını “içselleştirmiş” değildir (en azından proje tarama & indexing konusunda Cursor daha agresif). Yine de Copilot Enterprise planındaki “*Copilot için bilgi tabanları*” özelliği, şirket dokümanlarını ve rehberlerini AI'nın cevap havuzuna katarak benzer onboarding desteği sunmayı hedefliyor ⁶³. Uzun vadede, bu araçlar yeni geliştiricilerin çok daha hızlı üretken olmasını sağlayacak, belki de “aylarca kıdemli gözetimi” dönemini kısaltacaktır.

Kod Kalitesi ve Bakım: Üretkenlik artarken kod kalitesinin düşmemesi uzun vadede önemlidir. AI araçları burada çift taraflı etki edebilir. İyi kullanıldığında, Copilot ve Cursor **en iyi pratikleri teşvik eden kodlar üretebilir**, gereksiz hataları en baştan önleyebilir ve tutarlı kod stilini koruyabilir. Örneğin

Copilot'un test senaryoları önerme veya Cursor Bugbot'un potansiyel bug'ları işaretleme yetenekleri, kaliteyi artırır. Uzun vadede daha az üretim hatası ve daha yüksek kod standartları beklenir. Öte yandan geliştiricilerin AI'ın önerdiği kodları *anlamadan* kabul etme riski de vardır; bu risk, uygun kod inceleme süreçleriyle yönetilmelidir. Copilot ve Cursor, önerilerin sorumluluğunu nihai olarak insana bırakan araçlar olduğundan, ekip kültürü olarak AI'ın çıkardığı işi gözden geçirme disiplini oluşturulmalıdır. Bu sağlandığında, uzun vadede bu araçlar **tekrarlayan hataların azalması, kod tabanının daha derli toplu olması** (AI, benzer problemleri benzer şekilde çözdüğünden tutarlılık artabilir) gibi getiriler sağlayacaktır.

Takım Moral ve Yaratıcılık: Bir diğer gözlemlenen uzun vadeli etki de, geliştiricilerin iş tatmini ve yaratıcılığındaki değişimdir. Sıkıcı ve monoton işleri AI'a devredebilmek, mühendislerin daha çok değer katan işlere (sistem tasarımı, optimizasyon, yenilikçi özellik geliştirme gibi) odaklanmasına imkân verebilir. Fox CTO'su Melody Hildebrandt, Cursor'ın ekibindeki mühendisleri oldukça heyecanlandırıldığını, Slack üzerinden kendisine *"Bu teknolojiyi getirdiğiniz için teşekkürler!"* mesajları yağdığını aktarmıştır ⁸⁰. Nvidia CEO'su Jensen Huang ise şirketlerindeki ~40.000 mühendisin AI ile desteklenmesi sonucu **verimliliklerinin inanılmaz arttığını** vurgulayarak, Cursor'ı en sevdikleri kurumsal AI hizmeti olarak övmüştür ⁸¹. Bu anekdotlar, ekibe modern araçlar sunmanın motivasyon artırıcı etkisine dikkat çekiyor. Copilot'un da benzer şekilde geliştirici deneyimini iyileştirdiği, özellikle daha az tecrübeli elemanların daha özgüvenli katkı yapmasını sağladığı bilinmekte. Uzun vadede, AI asistanları "yapılması gereken ama geliştiriciye keyif vermeyen" işleri üstlenerek, insanlara daha yaratıcı veya eğlenceli kısımları bırakabilir. Bu da takım moralini ve iş kalitesini dolaylı olarak iyileştirebilir.

Özetle, **uzun vadede Copilot ve Cursor gibi araçlar şirketinizin yazılım üretme hızını ve kalitesini artırırken, ekibinizi daha güçlü kılar**. Cursor'ın agresif AI entegrasyonu, doğru adapte olduğunda, çarpıcı verimlilik sıçramaları getirebilir (daha kısa ürün döngüleri, kişi başı daha fazla çıktı, vs.). Copilot ise istikrarlı bir şekilde her geliştiricinin verimini yukarı çeker ve bunu geniş ölçekte yapmanızı sağlar. Hangisi seçilirse seçilsin, bu araçlardan maksimum faydayı görmek için ekibin adaptasyonuna yatırım yapmak ve AI'yı takım kültürünün bir parçası haline getirmek gerekir.

Kurumsal Özellikler: Güvenlik, SSO, Lisans Yönetimi ve Raporlama

Büyük ölçekli ekipler ve kurumsal ortamlarda, bir geliştirme aracının teknik yetenekleri kadar **yönetim, güvenlik ve uyumluluk özellikleri** de karar sürecinde kritiktir. Bu bölümde Cursor ve Copilot'un kurumsal tekliflerini bu açıardan kıyaslıyoruz.

Kimlik Doğrulama ve Tek Oturum Açma (SSO): Kurumsal güvenlik politikaları, kullanıcıların şirketin kimlik sağlayıcısı üzerinden doğrulanmasını şart koşar. **Cursor Teams ve Enterprise** planları, SAML 2.0 ve OpenID Connect (OIDC) protokollerini destekleyerek **SSO entegrasyonu** sunar ³⁸ ⁸². Yani şirketiniz Okta, Azure AD, Google Workspace, OneLogin vb. bir kimlik sağlayıcı kullanıyorsa, Cursor buna entegre olup kullanıcıları merkezi kimlik yönetimiyle senkronize edebilir. Hatta Cursor Enterprise, **SCIM protokolü** ile otomatik kullanıcı provizyonu/deprovizyonu desteği sunarak işe giren/çıkan personelin lisans yönetimini kolaylaştırır ⁸². GitHub Copilot Business/Enterprise ise GitHub platformunun kendisini kullandığı için, dolaylı bir SSO söz konusudur: Eğer şirketiniz GitHub Enterprise Cloud kullanıyorsa, zaten GitHub hesaplarını SSO ile kurum AD'sine bağlama imkânı vardır. Copilot da bu mekanizmanın üzerinde çalışır, yani kullanıcılar SSO ile GitHub'a girmişse Copilot'a da girmiş olur. Özetle **her iki ürün de SSO ile uyumludur**, ancak Cursor bunu doğrudan kendi servisinde sağlarken, Copilot için GitHub katmanında çözülür. Pratikte güvenlik ekipleri açısından her ikisi de yeterli entegrasyonu sunacaktır.

Lisans Yönetimi ve Erişim Kontrolü: Copilot for Business lisansları GitHub organizasyonunuza koltuk bazı olarak tanımlanır. Yöneticiler GitHub ayarlarından kimlere Copilot açılacağını atayabilir, diledikleri zaman kullanıcı ekleyip çıkarabilirler ⁸³. Ayrıca GitHub, kullanıcıların kendilerinin lisans talebinde bulunup onay almasına da imkân tanır (self-serve model). Copilot Enterprise planında, merkezi bir konsoldan alt organizasyonlara lisans dağıtma esnekliği de vardır ⁸⁴. Cursor'da ise **Merkezi Takım Faturalandırması** (centralized billing) özelliği ile tüm kullanıcıların lisansları tek bir hesapta toplanır ve yönetilir ⁴⁶. Admin birini davet ettiğinde, o kullanıcı Pro özellikleriyle takımın parçası olur. Teams planında kullanıcı sayısını artırıp azaltmak faturaya yansır, Enterprise planında ise özel anlaşma ile istenen esneklikte lisanslama yapılabilir (örneğin kullanım havuzu gibi) ⁴⁶. Cursor ayrıca **role-based access control (RBAC)** desteği ile yöneticilere bazı ince ayar olanakları verir: Örneğin takım içinde kimler "AI model ayarı yapabilir", kimler sadece belirli bir modeli kullanabilir gibi roller tanımlanabilir ³⁸. Copilot tarafında henüz bu tür detaylı rol tanımları bulunmuyor; ancak Copilot Enterprise'da "organization-wide policy management" özelliği ile bazı genel kural ve sınırlar belirlenebiliyor ⁶² (örneğin tüm organizasyonda "açık kaynak kod önerme" kapalı olsun, gibi). Genel olarak lisanslama ve erişim yönetimi konusunda her iki ürün de kurumsal ihtiyaçları adresliyor, fakat Cursor biraz daha ayrıntılı kontrol imkanı sunuyor diyebiliriz.

Kullanım Raporları ve Analitik: Ekip yöneticilerinin AI araçlarının kullanımını izleyebilmesi, hem değer ölçümü hem de denetim açısından önemli. **Cursor Teams** ile gelen yönetici paneli, takım üyelerinin yaptığı toplam istek sayısı, aktif kullanım süresi, vs. gibi metrikleri görsel bir arayüzde sunar ⁷⁰. Admin, bu sayede hangi birimin AI'ı daha çok benimsediğini, kimlerin hiç kullanmadığını veya kota limitlerine kimin sık takıldığını gözlemleyebilir. Bu veriler, iç eğitim planlamak veya ek lisans kapasitesi gerekip gerekmediğini anlamak için kullanılabilir. **Copilot Business** ise GitHub "Copilot Usage Metrics" adında bir bölüm sunar ⁸⁵ ⁸⁶. Buradan organizasyon bazında kaç öneri verilmiş, kaç kullanıcı aktif gibi temel istatistikler alınabilir. Copilot Enterprise planında daha gelişmiş metrikler ve **audit log** bulunur ⁶³. Audit log'larda kim hangi zamanda Copilot etkinliği yapmış, organizasyon politikasını ihlal eden bir şey olmuş mu gibi detaylar vardır. Ayrıca GitHub API'leri üzerinden bu kullanım verilerini çekip kendi BI (iş zekâsı) araçlarınıza entegre etmeniz de mümkündür. Sonuçta Cursor'ın paneli daha kullanıcı dostu ve dahili raporlama odaklı iken, Copilot'un yaklaşımı daha çok ham veriyi sunup yorumu size bırakmak şeklindedir. Kurumsal düzeyde her iki yaklaşım da işe yarar; ancak **Cursor'ın kullanım analitiği konusunda bir adım önde** olduğunu not edebiliriz (örneğin Cursor Enterprise, AI tarafından üretilen kodun takibini yapan bir API bile sunmayı planlıyor, bu sayede hangi kod satırının AI tarafından yazıldığını audit edebilirsiniz demekle ⁸⁷).

Güvenlik ve Uyumluluk (Compliance): Copilot ve Cursor, güvenlik standartları konusunda bazı benzerlikler ve farklılıklara sahip. **Sertifikasyonlar:** GitHub Copilot Enterprise, SOC 2 Type 1 raporu almıştır ve ISO 27001 sertifikalıdır ⁶⁰. Cursor da 2025 itibarıyla SOC 2 Type 2 sertifikası aldığını belirtmektedir ⁴². ISO veya diğer sertifikalar konusunda Cursor tarafında kamuya açık bilgi bulunmamakla birlikte, GDPR/CCPA uyumluluğu ve yıllık penetrasyon testleri yaptıkları ifade edilmiştir ⁴¹ ⁴². Büyük kurumladaysanız bu sertifikalar güvenlik değerlendirme süreçlerinde önem taşır. Copilot'un burada bir nebze daha kapsamlı belgelendirme sağladığı söylenebilir (ISO sertifikasyonu gibi). **Veri İşleme ve Gizlilik:** İki ürün de müşteri kodunun gizliliğine vurgu yapar, ancak yaklaşımları farklıdır. Cursor, hiçbir kullanıcı kodunun kalıcı olarak tutulmadığını ve ne kendi ne de model sağlayıcıların bu kodları tekrar eğitmek için kullanmadığını garanti eder ⁴⁰. Ayrıca "Privacy Mode" ile kullanıcıların bilerek ya da bilmeyerek hassas veri göndermesinin önüne geçer (bu mod açıkken, e-posta, anahtar, URL gibi tespit ettiği özel bilgileri modele iletmez). GitHub Copilot ise 2023 öncesi dönemde kullanıcı kodlarından anonim telemetri toplayıp model geliştirmede kullanabiliyordu; fakat Copilot for Business/Enterprise'da "Content Exclusion" özelliği sayesinde şirket kodlarınızın hiçbir şekilde kaydedilmemesini ve eğitimde kullanılmamasını seçebilirsiniz ⁶². Yine Copilot organizasyon politikalarında, AI önerilerinin ne oranda mevcut koda benzeyebileceğini, vs. belirleyebilirsiniz (örn: %50'den fazla aynıysa önerme gibi kurallar). Bu, potansiyel IP ve telif hakkı risklerini kontrol altına alır.

Veri Bölgeselliği: Copilot Enterprise, AB müşterileri için verileri Avrupa'da işleme seçeneği sunar ⁶¹. Cursor tarafında belirli bir veri merkezi seçeneği olduğuna dair bilgi yok; global bir hizmet olarak muhtemelen ABD merkezli sunuculardan çalışır. Bu, Avrupa kökenli çok sıkı regülasyonları olan kurumlar için bir fark yaratabilir.

Özel Dağıtım (On-Premises) ve VPC: Bazı kurumsal müşteriler, AI araçlarını kendi özel bulutlarında veya veri merkezlerinde çalıştırmak isteyebilir (özellikle finans, savunma gibi sektörlerde). Şu anda **GitHub Copilot, GitHub Enterprise Server üzerinde çalışmamaktadır**, yani tamamen bulut tabanlıdır ⁶⁹. Cursor ise sıkça sorulanlarda "On-prem veya VPC dağıtımı destekliyor musunuz?" sorusunu alıyor. Resmi dokümantasyonda halihazırda on-prem desteği olmadığı, ancak talebe göre değerlendirilebileceği ima edilmiştir (muhtemelen büyük bir müşteriyseniz, özel bir çözüm sunmaya istekli olabilirler). Dolayısıyla bu konuda iki ürün de tam isteneni veremeyebilir; ancak Cursor'ın esnek bir startup olarak çözüm üretme olasılığı daha fazladır.

Destek ve Eğitim: Kurumlar için bir diğer önemli konu, satın alınan ürün için alınabilecek destektir. Copilot Enterprise müşterileri, GitHub'ın premium destek kanallarına erişebilir ve Azure/GitHub müşteri temsilcilerinden yardım alabilir. Cursor Enterprise müşterilerine ise **öncelikli destek ve hesap yöneticisi** sağlandığı belirtiliyor ⁸⁷. Ayrıca Anysphere (Cursor'ın şirketi) büyük kurumsal müşterilere profesyonel hizmetler sunup, kendi ekiplerini eğitime veya AI'ı projelerine entegre etme konusunda danışmanlık yapabileceklerini ifade ediyor ⁸⁸. Bu, henüz AI araçlarını olgunlukla kullanmayan şirketler için faydalı olabilir. Microsoft tarafında da benzer şekilde Copilot'u kurum içinde yaygınlaştırma üzerine atölye çalışmaları ve rehberlik sağlayan **"AI Adoption"** ekipleri bulunmaktadır ⁷⁶.

Sonuç olarak, **kurumsal özellikler bakımından Copilot ve Cursor büyük ölçüde denk bir tablo sergiliyor**. Copilot, güvenlik uyumu ve global altyapı konusunda bir adım önde; Cursor ise özelleştirme, kullanım analitiği ve potansiyel esneklik konularında avantajlı. SSO, lisans yönetimi gibi temel beklentileri her ikisi de karşılıyor. Hangisinin seçileceği, şirketinizin hangi riskleri önemseydiğine ve hangi kontrol düzeyine ihtiyaç duyduğuna göre değişebilir. Örneğin **eğer SOC2/ISO gibi sertifikalar ve AB veri yerielliği kritikse Copilot, eğer AI kullanımını içeride daha fazla görünür kılmak ve modifiye etmek önemliyse Cursor** tercih sebebi olabilir.

Lisanslama, Maliyet ve Toplam Sahip Olma Maliyeti (TCO)

Şimdi iki ürünün lisanslama modellerini ve maliyetlerini karşılaştıralım. Doğrudan maliyet karşılaştırması yaparken, yalnızca abone ücretlerini değil, kullanım bazlı maliyetleri ve dolaylı TCO unsurlarını da göz önünde bulunduracağız. Aşağıdaki tabloda başlıca planlar ve fiyatlar özetlenmiştir:

Ürün / Plan	Fiyatlandırma	Notlar
Cursor Teams (Ekip Planı)	\$40 kullanıcı/ay (aylık ödeme) ⁴⁶ , \$32 kullanıcı/ay (yıllık ödeme) ⁸⁹ ⁴⁷	Pro plan tüm özellikler + ekip yönetimi. Aylık 500 hızlı istek kotası/kişi ⁹⁰ . Üstü kullanım için ek ücret gerekebilir ⁶⁷ . SAML SSO ve merkezi faturalandırma dahildir.
Cursor Enterprise	Özel (Custom) fiyatlandırma. ⁴⁶	Teams planına ek olarak: Kullanım kotaları ekip geneli havuzlanabilir, ek API/özellikler, öncelikli destek. SCIM entegrasyonu ile kullanıcı provizyonu ⁸² . Fiyat, kullanıcı sayısı ve istek hacmine göre şirketle görüşülerek belirlenir.

Ürün / Plan	Fiyatlandırma	Notlar
GitHub Copilot Business	\$19 kullanıcı/ay ⁶⁵ (yıllık ödemede değişebilir)	Sınırsız kod tamamlama, aylık 300 premium (GPT-4 vs.) istek hakkı/kişi ⁶⁶ . Merkezi yönetim, temel politika kontrolleri dahil. GitHub Enterprise Cloud veya Team ile kullanılabilir ⁹¹ .
GitHub Copilot Enterprise	\$39 kullanıcı/ay ⁶⁶	Business planının tüm özelliklerine ek olarak: aylık 1000 premium istek hakkı/kişi ⁶⁶ , içerik hariç tutma, organizasyonel ek politikalar, audit log'lar ⁹² ⁶³ . Büyük kurumlara yönelik genişletilmiş özellikler.

(Not: Yukarıdaki fiyatlar USD cinsindendir ve 2025 itibarıyla güncel alınmıştır. Döviz kuru ve bölgesel fiyatlandırma farklılıkları dikkate alınmalıdır.)

Tablodan görüldüğü üzere **Copilot Business, Cursor Teams'in liste fiyatının yaklaşık yarısıdır**. Cursor yıllık ödemede indirim yapsa da yine de %60 daha yüksek maliyetlidir. Copilot Enterprise ise fiyat olarak Cursor Teams'e yakındır, ancak Copilot Enterprise yalnızca çok büyük şirketlerin tercih edeceği ek uyumluluk özellikleri sunar. Çoğu şirket muhtemelen Copilot Business ile Cursor Teams arasında karar verecektir.

Kullanıma Bağlı Maliyetler: Önemli bir fark, Cursor'ın planlarında **kullanım limitleri** bulunmasıdır. Cursor Pro'da aylık 500 "hızlı" (fast) istek, sınırsız "yavaş" istek hakkı vardır ⁹³. "Yavaş istek" genelde arkaplanda uzun süren veya daha düşük öncelikli işlem demektir. Teams planı muhtemelen kullanıcı başına benzer limitler verir (UI Bakery kaynağı Business plan için "Pro'daki her şey" diyor, dolayısıyla 500 hızlı istek/ay/kişi) ⁹⁴. Bu limit aşılsa ek istek başına belli bir ücret (örn: \$0.04 gibi) faturalandırılabilir. Copilot Business da benzer şekilde premium istekleri 300 ile sınırlıyor; ancak buradaki premium istek kavramı daha çok GPT-4 gibi üst seviye modele giden istekler için. Standart modellerle (ör. GPT-3.5 Turbo veya GitHub'ın optimizasyonlu modeliyle) yapılan kod tamamlamalarına sınırsız diyebiliriz. Tipik bir kullanıcı normalde ayda yüzlerce GPT-4 seviyesinde talepte bulunmayacağı için, pratikte Copilot'un limiti daha az hissedilir. Bir karşılaştırmaya göre, Cursor Pro'da 500 istek limiti bazı kullanıcılara bir hafta yetmişken, Copilot'ta yoğun kullanımda dahi rate limitine nadiren takıldıklarını ifade etmişlerdir ²⁶. Yine de Copilot da Mayıs 2025'te premium istek sınırını getirip "daha fazla istek gerekiyorsa ek paket alın" şeklinde bir modele geçti (ek istek başına ~\$0.04) ²⁷, bu bakımdan iki ürün kısmen benzer yaklaşıma gidiyor. **Toplam sahip olma maliyeti hesabında**, eğer ekibiniz AI araçlarını yoğun ve etkin şekilde kullanırsa, Cursor'da temel abonelik ücretine ilave kullanım faturası çıkma ihtimali varken, Copilot'ta büyük oranda sabit bir bütçede kalabilirsiniz.

Dolaylı Maliyetler ve Tasarruflar: TCO hesabında sadece lisans ücreti değil, bu araçların getirdiği veya götürdüğü diğer maliyetler de önemlidir. Örneğin Cursor'a geçişin yol açacağı **geçici verim kaybı** da aslında parayla ölçülebilir: İlk haftalarda ekibin biraz yavaşlaması, belki de fazladan mesai veya kayıp fırsat olarak değerlendirilebilir. Copilot'ta böyle bir geçiş maliyeti yok denecek kadar azdır (VS Code'a eklenti kurmak dışında). Öte yandan, uzun vadede Cursor'ın sunduğu yüksek üretkenlik artışı, lisans maliyetini fazlasıyla telafi edebilir. Diyelim Cursor bir mühendisin aylık çıktısını %20 artırıyor – bu, o mühendisin maaşının %20'si değerinde ek iş anlamına gelir. Mühendis maaşları düşünülüğünde, \$40/ay ek maliyet bunun yanında küçük kalabilir. Benzer şekilde Copilot'un sağladığı kazanım, kendi maliyetinin kat kat üzerinde olabilir. Bu tür **ROI (yatırım getirisi)** analizleri şirketten şirkete değişecektir. Bir finansal hizmetler şirketi, "AI aracı seçerken ham ücretinden ziyade getirisiyle birlikte düşünmeli" diye not düşmüştü ⁹⁵. Yani belki daha pahalı bir araca geçmek, eğer getirdiği verimle çok daha fazlasını kazandırıyorsa mantıklı olabilir.

TCO Özet: Lisans maliyeti açısından Copilot avantajlı görünmekte, özellikle mevcut ekosisteme entegrasyon maliyeti de yoksa. Cursor ise birim fiyat yüksek olsa da tam potansiyeliyle kullanıldığında elde edeceğiniz kazanımlarla bu farkı kapatabilir. Risk, eğer Cursor alınır ama ekip onu tam benimseyemezse, o zaman sadece daha pahalı bir editör kullanmış olursunuz – bu istenmeyen bir senaryo. Copilot'ta ise adoption (benimsenme) düşük kalsa bile maddi kayıp daha sınırlıdır ve en azından mevcut düzeniniz bozulmamış olur. Bu açıdan, **Copilot düşük riskli, Cursor yüksek getirili (ama yüksek riskli) bir yatırım** gibi değerlendirilebilir. Toplam sahip olma maliyetinde de Copilot'un çevresinde halihazırda bir ekosistem olduğu (mevcut bulut sözleşmeleri, paket anlaşmalar vs. ile belki indirimler alınabildiği) unutulmamalı. Microsoft, GitHub ürün paketleri içinde Copilot'u cazip tekliflerle sunabilir, eğitim kredileri vs. verebilir. Cursor ise startup olduğu için liste fiyatı üzerinden gidecektir (çok büyük anlaşmalarda indirim olabilir ama şu an ölçek ekonomisi Copilot tarafında).

Aşağıda karar sürecini kolaylaştırmak için temel özellik ve maliyetlerin özetlendiği doğrudan karşılaştırmalı bir tablo sunulmuştur:

Özellik Kategorisi	Cursor IDE (Teams)	GitHub Copilot for Business
Editör Entegrasyonu	VS Code tabanlı bağımsız IDE (Cursor uygulaması). VS Code eklentileri ve ayarları desteklenir ¹ . Yeni bir uygulama olarak dağıtılır.	VS Code ve birçok mevcut IDE'ye eklenti olarak entegre olur ⁵³ . Mevcut geliştirme ortamı içinde çalışır, ek uygulama gerekmez.
Kod Tamamlama	Gelişmiş çok satırlı tamamlamalar; projenin tamamını göz önünde bulundurarak otomatik import ekleme, sonraki düzenleme yerine tahmin gibi akıllı özellikler ⁹⁶ ⁹⁷ .	Satır içi (inline) tamamlamalarda güçlü. Tek satır veya blok bazlı öneriler sunar. <code>Alt+]</code> / <code>Alt+[</code> ile alternatif önerilere geçme veya <code>Ctrl+Enter</code> ile çoklu öneri alma imkânı var ⁹⁸ ⁹⁹ .
Kod Üretimi / Komutla Kod	<i>Composer</i> özelliği ile doğal dil komutundan tüm bir kod parçacığını veya dosyaları üretebilir ¹⁸ . Proje yapısına uygun kod oluşturmada başarılı, çoklu dil aynı anda destekleniyor ¹⁸ .	Copilot Chat aracılığıyla istekle fonksiyon/çözüm üretebilir. Ayrıca Copilot CLI ile açıklamadan komut satırı kodu oluşturma imkânı vardır ¹⁰⁰ ¹⁰¹ . Temel olarak talep üzerine blok üretimi yapar, tüm proje yerine odaklandığı dosya/istek vardır.
Chat / Sohbet Desteği	Entegre Cursor Chat (<code>Cmd+L</code>), kontekst farkındadır , açık dosyaları ve eklenen klasörleri bilir ¹⁹ . Chat üzerinden doğrudan kod üzerinde değişiklik uygulanabilir. (Görsel içerik: Ekran görüntüsü veya tasarım da sohbetin parçası yapılabilir deniyor) ¹⁹ .	Copilot Chat (ön izleme), VS Code içinde ayrı bir panel. Kod açıklama, hata bulma, iyileştirme tavsiyesi almayı sağlar ¹⁰² . Chat geçmişi tutma, bağlam ekleme (dosya attach etme) özellikleri geliştiriyor ¹⁰³ ¹⁰⁴ . Cursor kadar devrimsel bir farklılık yaratmıyor, ama VS Code kullanıcıları için oldukça yeterli ve kullanışlı.

Özellik Kategorisi	Cursor IDE (Teams)	GitHub Copilot for Business
Çoklu-Dosya Desteği	<i>Composer</i> ile tüm proje genelinde arama-değiştirme yapabilir, ilgili tüm dosyalara dokunan refaktörler mümkün ³¹ . Ör: Bir sınıfı bölüp ilgili her dosyaya import ekleme gibi görevleri başarabilir ¹⁰⁵ .	Copilot Edits özelliği (preview), kullanıcı tanımlı “çalışma seti” içindeki birden fazla dosyada istenen değişiklikleri yapabilir ¹⁰⁶ . Ancak bazen yavaş veya hatalı dosya eşleştirmeleri olabiliyor ²⁰ . Agent Mode ile daha özerk çoklu dosya düzenlemeleri yolda.
Otonom AI Ajanı	<i>Cursor Agent</i> : <code>Cmd+.</code> ile aktif. Claude modelini kullanarak kod yazma, terminal komutları yürütme, testleri çalıştırma gibi çok adımlı görevleri otonom biçimde yapar ¹² ¹⁰⁷ . Derleme hatalarını görüp düzeltene dek iterasyon yapabilir. (Her işlem istek kotasından düşer) ¹² ¹⁰⁷ .	Copilot Agent Mode : 2025'te VS Code Insiders'da tanıtıldı ²² . Çok benzer şekilde, derleme/test çalıştırıp hataları gidererek hedef görevi tamamlamaya çalışıyor ¹⁶ ¹⁵ . Şu an ön izleme/ deneysel aşamada. Cursor kadar rafine olmayabilir ama kısa sürede genel kullanıma çıkması bekleniyor.
Kod İnceleme ve Hata Bulma	<i>Bugbot</i> eklentisi ile PR'ları tarar, muhtemel hataları listeler ve istenirse otomatik düzeltir ⁷¹ . Editör içinde de “Bug Finder” ile kodunuzda potansiyel sorunları bulup işaretleyebilir. (Takım planında sınırsız kullanılır, bireysel planda sınırlı) ¹⁰⁸ ¹⁰⁹ .	Copilot'un PR asistanı, PR değişikliklerini inceleyip öneriler yapar (şu an beta). <i>Copilot Code Review</i> özelliği, kodu yerel olarak inceleyip iyileştirme yorumları bırakabilir ⁵⁹ . Ayrıca Copilot, yazdığınız kodu anında analiz ederek uyarılar verebilir (şimdilik basic olsa da).
Özelleştirme Kuralları	<code>.cursorrules</code> dosyası ile proje özelinde AI'nın nasıl davranacağı tanımlanabilir (tercih edilen stil, yasaklı fonksiyonlar, vb.) ³⁷ . Model seçimi, parametre ayarları gibi gelişmiş kontrol imkanı sunar.	<code>.copilot-instructions.md</code> ile temel stil/yönerge verilebilir ⁶⁸ . Ayrıca <i>organizational policies</i> ile herkes için geçerli bazı kurallar konabilir (örn. şirket kodunu önerme/önermeme) ³⁹ . Ancak Cursor kadar kapsamlı değil, daha sınırlı ve yeni bir alan.
Commit Mesajları	AI ile otomatik commit mesajı oluşturur. Varsayılanlar biraz ayrıntılı/uzun olabilir, <code>.cursorrules</code> ile kısaltılmaları sağlanabilir ¹¹⁰ . Yine de her commit için zaman kazandırır.	PR oluştururken veya GitHub üzerinden commit atarken “ <i>generate commit message</i> ” butonu ile özet mesaj üretir. Copilot'un mesajları genelde öz ve anlaşılır oluyor ¹¹¹ . Lokal olarak VS Code içinden de benzer işlev mevcuttur.
IDE Entegrasyonu	Kendi IDE'si (Cursor) üzerinde çalışır. VS Code'dan geçtiğinizde alışmanız kolaydır, zira altyapı VS Code'un fork'u ¹ . Ancak VS Code dışında (JetBrains, vb.) bir editörde kullanamazsınız.	VS Code, Visual Studio, JetBrains, Neovim, Eclipse, Xcode gibi pek çok ortama kurulabilir ⁵⁴ ⁵⁷ . Yani farklı ekipler aynı AI yardımını kendi tercihlerine göre alabilir. Bu esneklik Copilot'un büyük artısı.

Özellik Kategorisi	Cursor IDE (Teams)	GitHub Copilot for Business
Desteklenen AI Modelleri	GPT-4 (OpenAI), GPT-3.5, Claude 2 (Anthropic Sonnet), vs. pek çok model seçeneği sunar. Hatta kendi hafif modeli (cursor-small) dahi var ³³ . Kullanıcı gerekli durumlarda model değiştirerek hız/kalite optimizasyonu yapabilir.	Copilot arka planda çeşitli modeller kullanıyor, son güncellemeyle model seçimi arayüze geldi: Örneğin “Claude 3.5 Sonnet, GPT-4.1, GPT-5, GPT-5 mini” gibi seçenekler görünmekte ¹¹² . Yine de Copilot bunları kendi altyapısıyla sunuyor, doğrudan API anahtarı vs. takviyesi yok.
Güvenlik & Gizlilik	SOC2 Type II sert., GDPR uyumlu ⁴¹ . Kodu sunucuda saklamaz ⁴⁰ . SAML SSO, içerik filtreleme (Privacy Mode) mevcut. İsteğe bağlı telemetri ayarları sunabilir. Henüz ISO sertifikası bilgisi yok. On-prem çözümü yok (ancak görüşülebilir).	SOC2 Type I, ISO27001 sert. mevcut ⁶⁰ . Kodu model eğitimi için kullanmaz (opt-out varsayılan Business’da). İsteğe bağlı AB veri işleme seçeneği ⁶¹ . SAML SSO (GitHub üzerinden), audit log, içerik hariç tutma gibi kurumsal özellikler Enterprise planında ⁹² ⁶³ . Tamamen bulut, on-prem yok.
Fiyat (özet)	~\$40 kullanıcı/ay (Teams) – kullanım sınırlarıyla ⁴⁶ ⁹⁴ . Kurumsal özel fiyat (muhtemelen kullanıcı başı biraz daha yüksek veya taahhülle indirimli). Not: Yıllık alımda ~%20 indirim var ¹¹³ .	\$19 kullanıcı/ay (Business) – neredeyse sınırsız kullanım ⁶⁵ . \$39 kullanıcı/ay (Enterprise) – ek özellikli. Not: Bireysel Pro plan \$10/ay olduğundan Copilot’un kurumsal fiyatları oldukça rekabetçi sayılır.

Yukarıdaki tabloya dayanarak, ekibinizin önceliklerine göre şu noktalar öne çıkabilir: Eğer **farklı IDE kullanan alt ekipler** varsa Copilot’un çoklu IDE desteği kritiktir. Eğer **herkesi tek bir süper güçlü AI ortamında toplamak** istiyorsanız Cursor’ın entegre yapısı çekici gelebilir. Bütçe kısıtı varsa Copilot bariz biçimde uygun maliyetlidir. Eğer **model esnekliği** ve şirket içi AI kontrolü istiyorsanız Cursor avantaj sağlar. Güvenlik/uyumlulukta Copilot kağıt üzerinde daha kuvvetli durmaktadır (daha fazla sertifika, global altyapı).

SWOT Analizi: Cursor IDE (Ekip Lisansı)

Güçlü Yönler (Strengths):

- **AI-Öncelikli IDE:** Yapay zekânın editöre derinlemesine entegre olması, çoklu dosya farkındalığı ve chat/agent özellikleriyle benzersiz üretkenlik imkânları sunuyor ²⁹ ¹⁸. Diğer araçlar eklenti iken Cursor baştan sona AI düşünülerek tasarlandı.
- **VS Code Uyumluluğu:** VS Code altyapısından çatallandığı için kullanım deneyimi tanıdık ve VS Code uzantıları, temalar, kısayollar büyük ölçüde destekleniyor ¹. Geçiş yapan ekipler alışık oldukları özellikleri yitirmiyor.
- **Gelişmiş Model Seçimi:** OpenAI GPT-4/3.5, Anthropic Claude 2, vs. birçok model arasından seçim yapabilme veya otomatik seçim politikaları uygulayabilme esnekliği var ³² ³³. Farklı senaryolarda en uygun modeli kullanarak kalite veya maliyeti optimize edebilir.

- **Çoklu Dosya ve Proje Geneli Refaktör:** Composer aracı tüm proje kodunu kapsayacak şekilde değişiklik yapabiliyor (dosya oluşturma, taşıma, kod ekleme) – büyük çaplı refaktör ve kod tabanı bakımı için güçlü bir özellik ³¹ .
- **Otonom Ajan Yeteneği:** Agent modu, kod yazma, test çalıştırma, hata düzeltme gibi ardışık işleri kendi tamamlayabilen bir “otonom geliştirici yardımcısı” gibi çalışıyor ¹² . Doğru kullanıldığında mühendis başına düşen iş yükünü ciddi azaltabilir.
- **Kurumsal Özellikler:** Teams planında SAML SSO, RBAC, merkezi loglama, org-wide privacy kontrolü, SCIM desteği gibi büyük şirketlere yönelik tüm özellikler mevcut ³⁸ ⁸² . Ayrıca SOC2 Type II, GDPR gibi uyumluluklara sahip olması kurumsal güven verir ⁴¹ .
- **Kullanıcı Topluluğu ve Hızlı İterasyon:** Cursor, hızlı büyüyen bir kullanıcı topluluğuna sahip. Forumu ve Slack kanallarında aktif geri bildirim alıyor. Küçük bir şirket olarak taleplere hızlı yanıt verip ürünü hızlı geliştirebilme avantajı var. Bu da kullanıcı isteklerinin ürüne çabuk yansması demek.
- **Yaratıcı ve Karmaşık İşlerde Üstünlük:** Özellikle sıfırdan modül oluşturma, birden fazla teknolojiyi entegre etme gibi kompleks işleri bir komutla yapabilmesi (ör. “React + Firebase ile uygulama oluştur” dediğinizde iskeleti çıkarması) yaratıcılığı hızlandırıyor. Bu tür kapsamlı görevlerde Cursor rakiplerinden önde değerlendiriliyor.

Zayıf Yönler (Weaknesses):

- **Yüksek Lisans Maliyeti:** Ekip lisansı kullanıcı başı \$40/ay ile pahalı. Üstelik istek kotası aşırsa ek maliyet çıkabiliyor ⁶⁷ . Copilot gibi rakiplere kıyasla iki kata varan maliyet dezavantajı var.
- **Kullanım Kotası ve Kısıtlar:** Aylık istek limitleri yoğun kullanımda ekipleri sınırlandırabilir. Özellikle Claude gibi modeller “fast request” kotasını hızlı tüketiyor. Bu, ağır kullanıcılar için can sıkıcı olabilir; limit bitince performans düşebilir veya ekstra ücret ödenir.
- **Yeni Araç Öğrenme Eğrisi:** VS Code temelli olsa bile ayrı bir uygulama. Ekiplerin alışması 1-2 hafta verim düşüşü yaratabiliyor ⁵ . Ayrıca AI-first akışa adapte olmak zaman alıyor. Değişime dirençli veya geleneksel yaklaşıma bağlı ekip üyeleri Cursor’ı tam verimli kullanamayabilir.
- **Kararlılık ve Olgunluk:** Ürün yeni olduğu için zaman zaman hatalar, çökme sorunları, VS Code’a göre geri kalmış özellikler olabiliyor. Örneğin VS Code’un son sürümlerindeki yeniliklerin Cursor’a geç gelmesi, extension uyumsuzlukları gibi riskler var. Büyük ölçekte test edilmiş bir araç olmaması kurumsal için risk.
- **Performans ve Kaynak Kullanımı:** Entegre AI özellikleri (özellikle büyük modellerle çalışırken) IDE’yi ağırlaştırabilir. 8 GB+ RAM önerisi zayıf makinelerde sorun teşkil edebilir ⁵¹ . VS Code’dan daha fazla CPU/RAM tüketimi gözlemlenebilir.
- **Ecosystem Lock-in:** Cursor’a geçilince Copilot veya benzer eklentilerden yararlanma imkânı azalıyor (teorik olarak Copilot eklentisi Cursor’a kurulabilir mi bilinmez, ama çift AI kullanımı pratik değil). Cursor aynı zamanda sadece kendi IDE’sinde çalışıyor, yani JetBrains kullanan biriyle standartlaşamazsınız. Bu bir anlamda kendi ekosistemine kilitlenme yaratabilir.
- **Model Tutarlılıkları:** Cursor’ın özellikle Claude tabanlı ajanı bazen çok iyi bazen zayıf performans gösterebiliyor ²⁶ . Bu tutarsızlık, geliştiricilerin her zaman güvenememesine yol açabilir. Bir gün otomatik çözdüğü sorunu başka gün çözemeyebilir.
- **Bulut Bağımlılığı:** Copilot gibi Cursor da internet bağlantısına ihtiyaç duyar. Fakat global bir servis olduğu ve belki OpenAI/Anthropic API kotalarına tabi olduğu için, yoğun anlarda yanıt gecikmeleri yaşanabilir. Kullanıcılar bazen “Cursor yanıt vermiyor” gibi durumlar bildirmiştir. Bu da üretkenliği kesintiye uğratabilir.

Fırsatlar (Opportunities):

- **Kurumsal Özelleştirme ve Self-Hosting:** Eğer Cursor gelecekte on-premise bir sürüm sunarsa, çok sıkı güvenlik isteyen kurumlar için benzersiz bir çözüm haline gelebilir. Şu an Copilot’un yapamadığını yaparak bankalar, hükümet kurumları gibi yerlere girebilir.

- **Model ve Özellik İnovasyonu:** Cursor, multi-model desteği sayesinde yeni çıkan güçlü modelleri (ör. GPT-5, Google Gemini vb.) rakiplerden hızlı entegre edebilir. Bu, sürekli en iyi modeli kullanarak kalite avantajı getirebilir. Yine eklenti ekosistemini de AI ile zenginleştirecek yenilikler (ör. AI tasarım asistanı, veri tabanı sorgu asistanı vs.) ek fırsatlar.
- **Yatay Yayılım:** Şu an geliştiricilere odaklı olan Cursor, ürün yönetimi, test ekipleri gibi diğer rollere de AI asistanları sunabilir. Örneğin "Cursor QA" aracı ile test otomasyon önerileri vermek vb. Bu genişleme, Cursor'ı yazılım ekiplerinin her alanında vazgeçilmez kılabilir.
- **Topluluk Desteği ve Büyüme:** Cursor zaten hype yakalamış durumda; eğer topluluk katkıları (extension, eğitim materyali vs.) artarsa, fiili standart haline gelme şansı var. Bazı open-source projeler Cursor konfigürasyonlarını dahil etmeye başlarsa yayılımı hızlanabilir.
- **Verimlilik Hikayeleri:** Cursor, başarılı vakaları belgelendirip yayınladıkça (case study), daha çok şirketi ikna edebilir. Özellikle rakiplerine kıyasla ROI'sini kanıtlarsa, premium fiyatına rağmen tercih edilebilir. Bu fırsat, müşterilerden ölçülebilir başarı hikayeleri toplamaktan geçiyor.
- **Partnerlikler:** Bulut sağlayıcıları veya büyük konsültasyon firmalarıyla iş ortaklıkları yaparak kurumsal kapıları daha kolay aralayabilir. Örneğin bir bulut IDE platformu (Replit vs) ile entegre olup orada default AI çözümü olmak gibi.

Tehditler (Threats):

- **Rekabetin Yakalaması:** GitHub Copilot hızla Cursor'ın ayırt edici özelliklerini kopyalıyor veya kendi versiyonunu sunuyor (agent mode, chat improvements vb.)²². Microsoft'un dev kaynaklarıyla, Cursor'ın avantajları uzun sürmeyebilir. Bu durumda Cursor'ın "neden ayrı bir IDE kullanayım?" sorusuna yanıtı zayıflar.
- **Pazar Doymunluğu ve Fiyat Baskısı:** AI destekli IDE pazarında JetBrains AI, Visual Studio IntelliCode, AWS CodeWhisperer gibi rakipler var ve sayıları artıyor. Büyük şirketler bu araçları paketlerinin içinde bedava sunabilir (VS aboneliklerine Copilot dahil edilmesi gibi kampanyalar olabilir). Cursor, bağımsız bir ürün olarak fiyat baskısı altında kalabilir, müşteri kazanmak için fiyat indirmek zorunda kalabilir.
- **Startup Riskleri:** Anysphere şirketi için finansal veya stratejik riskler mevcut. Örneğin beklenen gelir elde edilemezse hizmet sürekliliği riske girebilir. Büyük kurumsal müşteriler bir startup'a güvenip bağımlı olmak istemeyebilir. Ayrıca olası bir satın alma durumunda (örneğin Microsoft veya Google, Cursor'ı satın alırsa) ürünün yönü değişebilir, destek aksayabilir.
- **Regülasyonlar:** Kod üreten AI'lar için ileride yasal düzenlemeler gelebilir (özellikle telif hakları konusunda). Küçük bir şirket olarak Cursor'ın bu düzenlemelere uymak için kaynak ayırması zor olabilir, oysa Microsoft gibi devler lobicilik ve adaptasyon gücüne sahip. Yeni bir yasal engel, Cursor'ı piyasa dışında bırakma riski taşır.
- **Kullanıcı Kabulü:** Geliştiriciler arasında her ne kadar AI araçları popüler olsa da, bazı konservatif takımlar "yeni bir IDE'ye geçmeyi" zorlayıcı bulabilir. Cursor harika bile olsa, kullanım oranı düşük kalırsa ekipler dönüşüm için yatırım yapmaz. Yani kültürel bariyer, Cursor'ın yayılımını tehdit ediyor; Copilot mevcut düzeni bozmadığı için bundan daha az etkileniyor.

SWOT Analizi: GitHub Copilot for Business

Güçlü Yönler (Strengths):

- **Mevcut Ekosisteme Sorunsuz Entegrasyon:** Copilot, halihazırda yaygın kullanılan editörlere tak-çalıştır eklenerek benimsenmesi çok kolay bir çözüm. Şirket içi dev ortamını değiştirmeye gerek yok; bu da adaptasyonu hızlandırıyor⁸. Çoklu IDE desteği sayesinde farklı ekipler tek lisansla faydalanabiliyor⁵⁴.
- **Microsoft/GitHub Desteği ve Güvenilirlik:** Büyük bir teknoloji şirketinin ürünü olduğu için hizmet sürekliliği, ölçeklenebilirlik ve destek anlamında güven veriyor. SOC2, ISO 27001 gibi

sertifikaları tamamlamış olması ve global veri merkezi altyapısına sahip olması kurumsal müşteriler için kritik bir artı ⁶⁰ .

- **Uygun Maliyetli ve Sabit Fiyat:** Rakiplerine göre birim fiyatı düşük (Business: \$19/ay kullanıcı) ve kullanım sınırları gevşek. Bu sayede bütçe planlaması kolay ve küçük takımlar bile satın alabiliyor. Ayrıca bireysel kullanımda ücretsiz ve öğrenci/öğretmenlere zaten bedava sunulması, ekosistemde yaygınlığını artırarak fiili standart haline getiriyor.
- **Hızlı Üretkenlik Kazanımı:** Araştırmalar Copilot'un geliştirici hızına hemen olumlu etkisi olduğunu gösteriyor (ilk kullanımlardan itibaren %20-50 arası hız artışı raporları) ⁵⁶ . Bu somut kazanım, yöneticilere aracı savunmak için güçlü bir argüman sunuyor. Geliştiriciler de anında faydayı görünce aracı bırakmak istemiyorlar.
- **Geniş Kod Bilgisi ve Kaliteli Öneriler:** Copilot'un arkasındaki modeller (özellikle OpenAI Codex / GPT serisi) çok büyük ölçekte veriyle eğitilmiş durumda. Bu yüzden bir soruna çoğunlukla doğru veya en azından kabul edilebilir bir çözüm öneriyor. Önerilerin kalitesi kullanıcılarca genel olarak tatmin edici bulunuyor (kabul oranı ~%30 ve memnuniyet ~%90) ⁹ .
- **GitHub ile Derin Entegrasyon:** PR özetleri, otomatik commit mesajları, code review yorumları, issue özeti gibi GitHub platformu içindeki entegre AI özellikleri, Copilot deneyimini genişletiyor ⁵⁹ ¹¹¹ . Bu entegre deneyim, yazılım yaşam döngüsünün birden fazla aşamasında değer sağlıyor.
- **Kurumsal Yönetim Araçları:** Copilot Enterprise ile gelen policy yönetimi, audit log, content exclusion gibi özellikler şirketlerin risk ve uyumluluk kaygılarını gideriyor ⁹² ⁶³ . Ayrıca lisans yönetiminin GitHub ile entegre olması, bir yerde tüm ayarların toplanması kullanım kolaylığı getiriyor.
- **Sürekli İyileştirme ve Yol Haritası:** Microsoft ve GitHub, Copilot'a sürekli yeni özellikler ekliyor (örn: 2023-2025 arası chat, CLI, agent, model seçimi, vs. birçok yenilik geldi). Bu aktif geliştirme, ürünü güncel tutuyor ve rakiplerin avantajlarını kapatıyor. Örneğin Cursor'ın öne çıktığı agent özelliği 2025'te Copilot'a eklendi ²² .
- **Kullanıcı Topluluğu ve Eğitim Materyali:** Copilot yaygınlaştıkça çevresinde büyük bir topluluk oluştu. Bloglar, videolar, dersler ile kullanıcılar deneyim paylaşıyor. Ayrıca Stack Overflow gibi platformlarda Copilot'la ilgili binlerce soru-cevap var. Bu, yeni kullanıcıların öğrenmesini kolaylaştıran bir ekosistem demek.

Zayıf Yönler (Weaknesses):

- **Entegre Değil Eklenti Olması:** Copilot, her ne kadar işini iyi yapsa da nihayetinde bir eklenti. Bu nedenle editör deneyimini Cursor kadar kapsayıcı şekilde değiştirmiyor. Örneğin Cursor'da olduğu gibi AI için özel paneller, proje gezginine entegre akıllı özellikler gibi derin entegrasyonlar Copilot'ta sınırlı. Bu, AI'dan tam verim almak için bazen Chat paneli, bazen ayrı komutlar kullanmayı gerektiriyor, iş akışı tam anlamıyla AI-first olmuyor.
- **Sınırlı Otonomi (Şimdilik):** Copilot'un agent modu yeni ve henüz beta, dolayısıyla Cursor kadar özerk işlem yapma konusunda geride. Çok adımlı görevlerde halen insan yönlendirmesine ihtiyaç duyabiliyor. Kendisini *asistan* olarak konumladığı için kullanıcı müdahalesi olmadan proje yürütmesi henüz çok mümkün değil.
- **Az Özelleştirilebilirlik:** Copilot, "kutu içinden çıktığı gibi" kullanılıyor. Şirketinize veya projenize özgü kurallar öğretmek sınırlı. Örneğin yeni bir kütüphane geliştiriyorsanız, AI'a bunu tanıtıp ona göre tamamlama yaptıramazsınız (gerçi GitHub'ın "*private knowledge base*" özelliği bunu çözmeye çalışıyor). Cursor'da *.cursorrules* veya model seçimiyle özelleştirme daha fazla. Copilot'ta tek yapabildiğiniz, belki bir README veya yorum yazarak modele ipucu vermek.
- **Veri Gizliliği Algısı:** Copilot her ne kadar şirket içi kodu loglamasa da, bazı geliştiriciler "kodu bir dış servise yolluyoruz" konusunda hala tedirgin. Geçmişte Copilot'un lisanslı kod üretmesi tartışmaları zihinde kaldı. Cursor gibi "0 data retention" şeklinde açık bir iddiası yok (ama aslında

benzer bir politikası var). Bu algı, hukuk veya güvenlik ekiplerinin Copilot'a temkinli yaklaşmasına yol açabilir.

- **GitHub Bağımlılığı:** Copilot kullanmak için GitHub hesabı ve çoğu durumda GitHub Enterprise Cloud üyeliği gerekiyor. Bu, GitLab/Bitbucket kullanan veya kaynak kodlarını internette tamamen izole tutan şirketler için kullanım zorluğu demek. Cursor ise herhangi bir git deposuyla veya hiç internete çıkmayan kodla da (eğer on-prem olsaydı) çalışabilir. Copilot, GitHub ekosistemine olmayanları dışarda bırakabiliyor.
- **Destekli Diller ve Teknolojiler:** Copilot'un genel modeli hemen her dilde çalışsa da derinliği tüm dillerde eşit değil. Özellikle çok yeni veya niş diller/çerçeveler (mesela bir FPGA dili veya endüstriyel özel diller) için modeli yeterli bilgiye sahip olmayabilir. Cursor ise model esnekliği sayesinde belki o dile uygun bir model seçebilir (ancak bu da kısıtlı bir avantaj). Yine de Copilot'un "her şeyi biraz bilen" yapısı, "bazı şeyi mükemmel bilen" özel modellere kıyasla bazen yetersiz kalabilir.
- **Kotanın Algılanmaması:** Copilot Business'ın premium istek sınırı (300/ay) kurumsal kullanıcılara pek duyurulmadı ve birçok kullanıcı "sınırsız" sanıyor. İleride bu limite takılmaya başlarsa (daha çok agent mode ile olabilir), memnuniyetsizlik yaratabilir. Bu da bir zayıflık, çünkü kullanım artışı ile ek satın alma gerekebilecek. Şu an belirsiz bir alan.
- **Arayüz Sınırlamaları:** Copilot'un VS Code arayüzü ufak tefek aksaklıklara sahip olabiliyor. Örneğin Chat penceresinde uzun yanıtları yönetmek, kod bloklarını kopyalamak vs. bazen kullanışsız olabiliyor (özellikle ilk sürümlerinde böyle geribildirimler vardı). Cursor'ın kendi arayüzü AI etkileşimleri için daha özenli tasarlanmış durumda. Bu, kullanıcı deneyimi açısından Copilot'un ufak bir eksi noktası.

Fırsatlar (Opportunities):

- **Enterprise Ürün Paketi:** Microsoft, Copilot'u GitHub Enterprise, Azure DevOps gibi ürünlerle paketleyip çapraz satış fırsatlarını kullanabilir. Örneğin "*GitHub Advanced Security + Copilot*" indirim paketi sunarak kurumsal pazarda rakiplerini ekarte edebilir. Bu tür bundling, Copilot'un kullanımını patlatabilir.
- **Dikey Uzmanlaşma:** Copilot'un hali hazırda genel modelleri var, ancak belli alanlara özelleşmiş AI'lar (ör. Copilot for Data Science, Copilot for DevOps gibi) ortaya çıkarabilirler. Şirket içinde farklı rollere özel AI asistanları, Copilot markasını güçlendirebilir ve kullanım alanını genişletebilir.
- **Verimlilik Odaklı Satış:** Copilot'un değerini kanıtlayan daha çok araştırma veya vaka analizi yayınlayarak, henüz ikna olmamış şirketleri kazanma fırsatı var. Özellikle büyük danışmanlık şirketleri (Accenture, etc.) Copilot kullanımını verimlilik katalizörü olarak tavsiye etmeye başlarsa pazar payını artırabilir.
- **Daha Derin IDE Entegrasyonu:** Microsoft'un VS Code ekibi, Copilot'u daha da entegre edebilir (nitekim VS Code'un kendisine Copilot özellikleri eklemeye başladılar). Örneğin editörde bir hatanın üzerinde AI çözüm önerisi butonu belirmesi gibi özellikler, Copilot'u iyice görünmez ama vazgeçilmez bir yardımcıya dönüştürür. Bu fırsat, Cursor gibi ayrı bir araca ihtiyacı tamamen ortadan kaldırabilir.
- **Üçüncü Parti Geliştirici Desteği:** GitHub, Copilot eklenti ekosistemine dış geliştiricilerin katkısını açtı (Copilot Extensions kavramı). Bu sayede farklı AI sağlayıcıları veya özel yetenekler, Copilot arayüzüne entegre edilebilir. Örneğin şirketiniz kendi GPT-4 API'sini Copilot ile kullanmak için bir eklenti geliştirebilir. Bu esneklik Copilot'u platform haline getirme fırsatı sunuyor.
- **Yeni Modeller ve Optimizasyonlar:** OpenAI'nin ve diğer AI lab'ların sürekli daha iyi modelleri çıkıyor. Microsoft bu ortaklıkları güçlü tutarak Copilot'un "en iyi modelle çalışan" araç olmasını sürdürebilir. Örneğin GPT-5 veya özel bir "GitHub GPT" eğitilirse, Copilot kalitesi rakipsiz seviyeye çıkabilir.
- **Farklı Programlama Aksiyonları:** Copilot şu an kod yazmaya odaklı ama belki ileride görsel düzenleme (UI tasarımını değiştirme), proje yönetimi (Jira tiketi kapatma gibi) entegrasyonları ile

geliştiricinin daha fazla işini kolaylaştırabilir. Bu genişleme, Copilot'u yazılımcının tek asistanı haline getirme potansiyeli taşır.

Tehditler (Threats):

- **Alternatif AI Asistanları:** Piyasada Google'ın Codey/Studio Bot'u, Amazon CodeWhisperer, Anthropic Claude, Replit Ghostwriter gibi pek çok rakip belirdi. Özellikle CodeWhisperer AWS müşterilerine ücretsiz sunularak Copilot'un pazarını tehdit ediyor. Google da Android Studio gibi popüler ortamlarına kendi AI asistanını ekliyor. Bu rekabet, Copilot'un büyümesini sınırlayabilir.
- **Ekosisteme Bağımlılık Eleştirisi:** Copilot'un GitHub'a sıkı entegre olması, EU gibi bölgelerde antitröst tartışmalarına konu olabilir (Microsoft'un kendi platformunu kayırması vs.). Eğer düzenleyiciler Copilot'un GitHub ile paketlenmesini engeller veya kısıtlarsa, pazardaki avantajı azalabilir.
- **Telif Hakları ve Hukuki Riskler:** Copilot'un önerdiği kodların lisans ihlali olabileceği iddiasıyla açılan davalar mevcut. Eğer mahkeme kararıyla Copilot'un işleyişine kısıtlama gelirse (örneğin belli lisanslı kodlardan eğitim aldığı için tazminat veya modele müdahale kararı), hizmet kesintileri veya kalitesinde düşüş riski doğabilir. Bu tür hukuki belirsizlikler kurumsal müşterileri de endişelendirir.
- **Model Erişilebilirliği:** OpenAI gibi tedarikçiye bağımlı olmak da bir tehdit olabilir. Eğer OpenAI API maliyetlerini artırırsa veya Microsoft ile anlaşmazlık çıkarsa, Copilot'un maliyeti artabilir ya da model kalitesi düşebilir. Microsoft bir noktada kendi modelini tamamen geliştirmezse bu bir dışa bağımlılık riski.
- **Geliştirici Direnci:** Bazı geliştirici toplulukları AI asistanlarını sıcak karşılamıyor (iş güvencesi endişesi, kod kalitesi endişesi gibi). Eğer ileride AI karşıtı bir hareket güçlenir ve şirketler bunun baskısıyla AI kullanımını kısıtlarsa Copilot bundan etkilenecektir. Örneğin bazı açık kaynak projeler, Copilot kullanan katkıcıları kabul etmeme gibi politikalar düşünmüştü.
- **Platform Değişimleri:** Yazılım geliştirme araçları hızla evriliyor. Örneğin web tabanlı IDE'ler (GitHub Codespaces vb.) yaygınlaşırsa, buralarda Copilot rekabet avantajını korumalı. Ya da yepyeni bir programlama paradigması çıkarsa (düşük kod/no-code platformların yükselişi gibi), Copilot'un önemi azalabilir. Bu tür makro değişimler, klasik kod tamamlama asistanlarının değerini düşürebilir.

SWOT analizlerinden görüleceği üzere, her iki ürünün de güçlü yanları fazla, ancak riskleri ve pazardaki meydan okumaları da mevcut. Karar verirken bu iç ve dış faktörleri dikkate almak gerekecektir.

Sonuç ve Öneri (Kurumsal Karar Özeti)

Hangi aracı seçmenin şirketimiz için daha doğru olacağı, ekibimizin ihtiyaçları, kültürü ve öncelikleri doğrultusunda değerlendirilmelidir. Aşağıda karar vermeyi kolaylaştıracak özet maddeler ve öneriler sunulmuştur:

- **Mevcut Durum ve Alışkanlıklar:** Halihazırda VS Code kullanan ve Copilot deneyimi olan bir ekibiz. Eğer ekibimiz genel anlamda mevcut düzeninden memnunsa ve büyük bir değişiklik için motivasyon yoksa, **GitHub Copilot for Business ile devam etmek** en düşük riskli seçenek olacaktır. Copilot, tanıdık ortamımıza zahmetsizce entegre olduğundan, verimlilik kazancımız kesintiye uğramadan sürecektir. Üstelik yeni Copilot özellikleri (Chat, Agent mode vb.) mevcut üretkenlik seviyemizi daha da yukarı taşıyabilir.
- **Üretkenlik Potansiyeli ve Hedefleri:** Eğer önümüzdeki dönemde yazılım geliştirme hızımızı **radikal şekilde artırma** hedefimiz varsa, ve ekibimiz AI-first çalışma modelini benimsemeye istekliyse, **Cursor IDE dikkate değer bir opsiyondur**. Cursor, doğru uygulandığında ekibimizin

yazılım üretim şeklini dönüştürüp belirgin sıçramalar sağlayabilir (daha kısa release döngüleri, mühendis başına daha fazla çıktı gibi). Ancak bu kazançların ancak **araç iyice benimsendiğinde** geleceğini unutmamalıyız – yani Cursor’a geçiş bir “sihirli değnek” olmayacak, başlangıçta yatırım ve eğitim gerektirecek. Eğer bu yatırımı yapmaya hazır ve kararlıysak, orta vadede Cursor’dan elde edilecek kazanımlar yüksek olabilir.

- **Maliyet ve ROI:** Bütçe kısıtımız ve üst yönetimin onay süreci düşünüldüğünde, Copilot’un **daha hesaplı** olduğu aşikâr. Copilot Business’a geçmek, lisans başı maliyet olarak Cursor’dan ciddi şekilde daha ucuz ve öngörülebilir ⁶⁵ ⁴⁶ . Öte yandan, Cursor’ın yüksek lisans maliyeti, eğer verimlilik artışıyla dengelenirse makul görülebilir. Burada kendi ekibimiz için bir **ROI analizi** yapmamız faydalı olur: Cursor kullanan mühendislerin %X daha verimli olacağını varsayarsak, bu iş gücü kazanımı parasal olarak lisans farkını karşılıyor mu? Eğer karşılamıyorsa, finansal açıdan Copilot kalmak mantıklı. Eğer Cursor’ın getirisi lisans farkını katbekat aşıyorsa, o zaman yatırım yapılabilir. Bu noktada pilot bir deneme belki metrik toplamak için değerlendirilebilir.
- **Ekip İçi Anket ve Geri Bildirim:** Karar öncesi ekibimizin fikrini almak da önemli. Geliştiricilerimizin bir kısmı belki Cursor’ı halihazırda denemiş ve şahsi tecrübeleri oluşmuş olabilir. Örneğin bazı mühendisler Cursor’ı sevip evde kullanıyor, bazıları ise VS Code’den kopmak istemiyor olabilir. İçeriden gelecek görüşler, kararın kabul edilebilirliğini de etkileyecektir. Şirketimizde Cursor’a hevesli bir kesim varsa, bunlarla Copilot memnurları arasında bir **ortak değerlendirme toplantısı** yapıp artıları eksileri tartışabiliriz. Genelde değişime açık, AI meraklısı geliştiriciler Cursor’ın imkanlarını daha iyi kullanırken, geleneksel yaklaşanlar Copilot’la daha konforlu olabiliyor.
- **Kademeli Geçiş İhtimali:** Karar illa siyah-beyaz olmak zorunda değil. İstersek ekibin bir bölümünde (örneğin R&D veya istekli bir pilot takımda) Cursor’ı deneyip, diğerlerinde Copilot kullanmaya devam edebiliriz. 3-6 aylık bir pilot program sonunda somut verilerle hangisinin bize daha çok uyduğunu görebiliriz. Bu **“hybrid kullanım”** yaklaşımı riskimizi azaltır ve herkesin görüş oluşturmalarını sağlar. Örneğin yeni projeye başlayan bir alt ekibi Cursor IDE’ye geçirip, mevcut kritik projelerde Copilot’la devam etmek gibi bir plan yapılabilir. Pilot sonunda eğer Cursor bariz avantaj sağlıyorsa tüm ekibe yayarız, yok eğer fark azsa veya verimsizse Copilot’ta kalırız.
- **Uzun Vadeli Stratejik Yönelim:** Şirket olarak uzun vadede mühendislik kültürümüzü nereye taşımak istediğimize bakmalıyız. AI’ı her süreçte kullanan, en yeni teknolojileri denemeye istekli “öncü” bir kültür mü, yoksa stabilite ve az risk almayı seven “temkinli” bir kültür mü olmak istiyoruz? Cursor’a geçiş ilk yolu, Copilot’ta kalmak ikinci yolu temsil ediyor denebilir. Eğer rakiplerimizin çoğu Copilot kullanacak ve biz Cursor ile farklılaşacaksak, bu bir stratejik cesaret örneği olabilir ve belki üstünlük de sağlayabilir. Tersine, herkes Cursor’a geçerken biz geride kalmamak için hamle yapmalıyız diyorsak, o da bir bakış açısı. Şu anki trend, Copilot’un yaygın standart olduğu yönünde, Cursor ise hızlı büyüyen bir niş. Hangi trende katılacağımızı seçmek, şirket vizyonumuzla da uyumlu olmalı.

Öneri: Genel değerlendirmede, **GitHub Copilot for Business** mevcut ekibimizin ihtiyaçlarını **daha düşük maliyetle** ve **daha düşük geçiş riskiyle** karşılıyor gibi görünmektedir. Halihazırda Copilot’tan memnun olan ve VS Code ekosistemine bağlı ekibimiz için, Copilot’u kullanmaya devam etmek mantıklı bir seçim olacaktır. Özellikle kısa vadede kesintisiz üretkenlik artışı ve kolay yaygınlaştırma açısından Copilot avantajlıdır. Bununla birlikte, Cursor IDE’nin getirdiği yenilikçi yaklaşımları tamamen göz ardı etmek de istemeyiz. Bu yüzden, **kısıtlı çaplı bir Cursor pilot projesi** yürütüp kendi deneyimimizle verimliliğini ölçmek akıllıca olacaktır. Pilot sonuçlarına göre ileride daha bilinçli bir karar verebiliriz. Şu

aşamada, şirket genelinde bir geçiş kararı vermek için Copilot'un henüz ciddi bir eksisini yaşamadık, aksine memnuniyet yüksek.

Dolayısıyla **kısa vadede önerimiz**, Copilot for Business lisanslarını ekibimizde aktif ederek tüm geliştiricilerimizin bu aracı günlük işlerinde kullanmasını sağlamak ve bu sırada gönüllü bir alt grubun Cursor IDE'yi denemesine olanak tanımaktır. Elde edeceğimiz geri bildirim ve metriklerle, **orta vadede stratejimizi yeniden değerlendirebiliriz**. Bu yaklaşım, ne mevcut düzeni bozarak riske atar ne de yenilik fırsatını kaçıır.

Son tahlilde, her iki aracın da ekip performansımıza katkı yapacağı açıktır. Önemli olan, bizim koşullarımızda **hangi aracın daha fazla değer katacağını** objektif şekilde ölçümleyebilmektir. Bu rapordaki bulgular ışığında, ilk adım olarak Copilot ile "bildiğimiz yoldan" ilerlemeyi, ikinci adım olarak Cursor ile "yeni ufukları" keşfetmeyi planlıyoruz. Böylece şirketimiz için en optimal AI destekli gelişim ortamını sağlama hedefimize emin adımlarla yaklaşacağız.

Kaynaklar: Çalışma boyunca belirtilen tüm bilgiler, güncel ürün dokümantasyonlarından, güvenilir teknoloji bloglarından ve kullanıcı geri bildirimlerinden derlenmiştir. Öne çıkan kaynaklar arasında GitHub ve Cursor resmi dokümanları, augmentcode.com ve builder.io üzerinde yayımlanan 2025 tarihli karşılaştırma incelemeleri, Medium ve Reddit üzerindeki kullanıcı deneyimleri yer almaktadır. İlgili kaynaklara numaralandırılmış biçimde metin içinde referans verilmiştir. Lütfen daha detaylı bilgi için bu referanslardaki içeriklere başvurunuz.

1 12 13 18 19 20 21 30 31 33 35 36 37 53 54 59 65 68 71 72 96 97 98 99 100 101 102 103

104 105 106 107 110 111 Cursor vs GitHub Copilot: Which AI Coding Assistant is better?

<https://www.builder.io/blog/cursor-vs-github-copilot>

2 3 4 5 6 7 8 9 10 11 17 29 39 50 51 52 55 56 58 60 61 64 95 JetBrains AI vs GitHub

Copilot vs Cursor: IDE-Native AI Tools Face-Off - Augment Code

<https://www.augmentcode.com/guides/jetbrains-ai-vs-github-copilot-vs-cursor-ide-native-ai-tools-face-off>

14 15 16 Introducing GitHub Copilot agent mode (preview)

<https://code.visualstudio.com/blogs/2025/02/24/introducing-copilot-agent-mode>

22 26 27 28 48 49 67 GitHub Copilot vs Cursor in 2025: Why I'm paying half price for the same features : r/GithubCopilot

https://www.reddit.com/r/GithubCopilot/comments/1jnboan/github_copilot_vs_cursor_in_2025_why_im_paying/

23 24 25 32 34 40 41 42 43 44 45 74 75 77 79 80 81 88 Enterprise · Cursor

<https://cursor.com/enterprise>

38 46 73 82 87 108 109 Pricing · Cursor

<https://cursor.com/pricing>

47 70 89 90 93 94 113 Cursor AI Pricing Explained: Which Plan is Right for You? | UI Bakery Blog

<https://uibakery.io/blog/cursor-ai-pricing-explained>

57 62 63 66 69 76 83 84 91 92 112 Plans for GitHub Copilot - GitHub Docs

<https://docs.github.com/en/copilot/get-started/plans>

78 85 86 Tracking license activation and initial usage with Copilot usage metrics - GitHub Docs

<https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/assign-licenses/track-usage-and-adoption>